

操作系统

Operating Systems

MS Roslyn

2026 年 10 月 9 日

摘要

本教程系统讲解操作系统的核心机制：从系统调用与内核态切入，深入进程与线程、CPU 调度、并发同步与死锁、内存管理与虚拟内存、文件系统与 I/O、以及保护与安全。教程强调「抽象 + 机制 + 策略」三条主线，用可计算的例子说明上下文切换开销、调度指标、页面置换、磁盘寻道等核心问题，帮助读者建立贯穿「硬件-内核-应用」的完整系统视角。

目录

第一章	操作系统概述与系统调用	6
1.1	操作系统做什么	7
1.1.1	操作系统的发展历程	7
1.2	内核结构	8
1.3	用户态与内核态	9
1.3.1	特权指令	9
1.3.2	模式切换与双模式的意义	9
1.4	系统调用	10
1.4.1	陷入 (trap) 机制与参数传递	10
1.4.2	系统调用的类别	10
1.4.3	实例: read 的完整流程	11
1.5	中断、异常与陷阱	12
1.5.1	三者的区别	12
1.5.2	中断处理流程	13
1.5.3	时钟中断与抢占	13
1.6	上下文切换的开销	14
1.7	本章小结	15
1.8	常见误区	15
第二章	进程与线程	16
2.1	进程概念	17
2.1.1	程序与进程	17
2.1.2	进程的组成	17
2.1.3	并发与并行	18
2.2	进程状态模型	18
2.2.1	五种状态	18
2.2.2	状态转换与触发事件	19
2.2.3	状态转换的方程表示	19
2.3	进程控制块 PCB	20
2.3.1	切换时的保存与恢复	20
2.4	上下文切换	21
2.4.1	切换步骤	21
2.4.2	开销来源	21
2.4.3	数值例题: 上下文切换开销	22

2.5	调度队列与调度器	23
2.5.1	就绪队列与等待队列	23
2.5.2	调度器的层次	23
2.6	线程	24
2.6.1	进程与线程的对比	24
2.6.2	线程的地址空间	24
2.6.3	用户级线程与内核级线程	24
2.6.4	多线程模型	25
2.7	进程间通信简介	25
2.8	本章小结	25
2.9	常见误区	26
第三章	CPU 调度	27
3.1	调度指标	28
3.2	先来先服务 FCFS 与最短作业优先 SJF	28
3.2.1	先来先服务 FCFS	28
3.2.2	最短作业优先 SJF	29
3.2.3	最短剩余时间优先 SRTF	29
3.3	时间片轮转 RR 与优先级调度	30
3.3.1	时间片轮转 RR	30
3.3.2	时间片大小的影响	30
3.3.3	优先级调度	31
3.4	多级反馈队列 MLFQ	31
第四章	同步与并发	33
4.1	临界区问题	34
4.1.1	三个必要条件	34
4.1.2	Peterson 算法	34
4.2	信号量	35
4.2.1	P/V 操作	35
4.2.2	生产者-消费者问题	35
4.2.3	读者-写者问题	36
4.3	管程与条件变量	36
4.4	死锁	37
4.4.1	四个必要条件	37
4.4.2	资源分配图	37
4.4.3	银行家算法	38
第五章	内存管理	39
5.1	地址空间与重定位	40
5.1.1	逻辑地址与物理地址	40
5.1.2	MMU 与基址/界限寄存器	40
5.2	分页	41

5.2.1	页、帧与页表	41
5.2.2	页表项 PTE	41
5.3	分段	42
5.3.1	段表与保护	42
5.3.2	分段与分页对比	42
5.3.3	段页式	42
5.4	TLB 与有效访问时间 EAT	43
5.4.1	快表 TLB	43
5.4.2	有效访问时间	43
第六章	虚拟内存	44
6.1	按需分页与缺页处理流程	45
6.1.1	地址翻译与页表项	45
6.1.2	缺页处理流程	46
6.2	页面置换算法: FIFO / LRU / 最优 OPT	47
6.2.1	三种算法的直觉	47
6.2.2	帧数 = 3 的逐步模拟	47
6.3	Belady 异常	49
6.4	抖动与工作集模型	50
6.4.1	抖动 (Thrashing) 的成因	50
6.4.2	工作集模型	50
第七章	文件系统	51
7.1	文件系统布局	52
7.2	inode 与多级索引	52
7.3	分配方式	53
7.3.1	三种方式的取舍	53
7.4	空闲空间管理	54
7.5	日志与崩溃一致性	54
第八章	I/O 与设备管理	56
8.1	I/O 软件层次	57
8.2	I/O 控制方式	57
8.2.1	轮询 (程序查询方式)	57
8.2.2	中断驱动 I/O	58
8.2.3	DMA (直接存储器访问)	58
8.3	磁盘结构与磁盘调度	58
8.3.1	常见磁盘调度算法	58
8.3.2	寻道长度计算示例	59
8.4	SSD 与闪存	59
8.4.1	FTL 与磨损均衡	60
8.4.2	与 HDD 对比	60

第九章	保护与安全	61
9.1	保护目标与域	62
9.2	访问矩阵、ACL 与能力表	62
9.2.1	访问矩阵	62
9.2.2	ACL 与能力表	63
9.3	UNIX 权限模型	63
9.3.1	rxw 与八进制表示	63
9.3.2	目录权限的特殊性	64
9.3.3	setuid、setgid 与 sticky	64
9.4	隔离与虚拟化	65
9.4.1	保护环	65
9.4.2	沙箱	65
9.4.3	虚拟化与容器	66

第一章 操作系统概述与系统调用

操作系统是「管理硬件、服务程序」的系统软件，同时是硬件与应用之间的抽象层与资源仲裁者。本章建立「内核结构-双模式-系统调用-中断」四块基石。

1.1 操作系统做什么

操作系统（Operating System, OS）是对计算机硬件的第一层、也是最基础的一层软件封装。它既把「裸机」的复杂细节隐藏起来，又承担着在众多竞争的程序之间分配资源的职责。可以从三个互补的视角来理解它的职能：

- **资源管理者**：CPU、内存、磁盘、外设在任何时刻都只能被有限个程序使用，OS 负责分配、回收与仲裁，使多道程序能够安全、公平、高效地并发推进。
- **抽象提供者**：把物理 CPU 抽象成**进程/线程**，把物理内存抽象成**地址空间**，把磁盘扇区抽象成**文件**，把设备控制器抽象成**统一的设备接口**。抽象让程序员不必直面硬件，也使软件具备可移植性。
- **控制程序**：负责保护与隔离。它需要防止应用程序越权访问硬件或彼此的内存，并在程序出错时把损害限制在局部。

三者并不矛盾：抽象使资源更易使用，管理使资源分配有序，控制保证系统不被滥用。可以把它概括为一句话——

OS = $\underbrace{\text{抽象}}_{\text{更好用}} + \underbrace{\text{管理}}_{\text{更有序}} + \underbrace{\text{保护}}_{\text{更安全}}.$ (1.1)

1.1.1 操作系统的发展历程

OS 的形态由「硬件条件」与「使用需求」共同塑造，大体经历了三个阶段。早期计算机昂贵而 CPU 相对稀缺，于是人们希望让 CPU 尽量「不空闲」，诞生了**批处理**；当用户希望机器像电话一样即时响应时，出现了**分时**；当计算必须与外部物理过程同步、结果迟到即失效时，产生了**实时系统**。

表 1.1: 操作系统发展的三个阶段

阶段	核心目标	交互方式	关键特征
批处理	提高 CPU 与作业吞吐率	成批提交，无交互	单道 → 多道批处理，作业排队、通道与中断引入
分时	公平、快速响应多用户	终端交互、时间片轮转	响应时间 R 可预期，同时性/独立性/及时性/交互性
实时	在硬性时限内完成	事件驱动、优先级抢占	分硬实时/软实时，强调 可预测性 而非单纯快

多道批处理的核心思想是：当一道作业等待 I/O 时，立即切换到另一道作业，使 CPU 与 I/O 设备尽可能并行工作，从而提升系统吞吐率。分时系统则把 CPU 时间切成**时间片**轮转分配给各终端用户，代价是频繁的上下文切换（见第 1.6 节）；实时系统最关心「最坏情况」是否满足截止期，因此常用优先级抢占与可预测的调度策略。

1.2 内核结构

内核（kernel）是 OS 中最核心、常驻内存、运行于特权态的部分。按「哪些功能放在内核态、哪些放到用户态」的不同取舍，形成了三种主流结构。

- 单体内核（Monolithic Kernel）：**文件系统、内存管理、设备驱动、网络协议栈等几乎所有服务都运行在内核态、共享同一地址空间，模块之间直接函数调用。优点是调用路径短、性能高；缺点是耦合紧，任一部分出现缺陷都可能使整个系统崩溃，且模块边界模糊、维护困难。Linux 属于此类（尽管其支持可动态加载的内核模块）。
- 微内核（Microkernel）：**内核中只保留最基本的机制——进程调度、地址空间、进程间通信（IPC）与中断处理；文件系统、驱动、网络等服务被移到用户态，作为服务器进程运行。优点是内核小、可靠性与可扩展性好、易移植；缺点是服务之间必须通过 IPC 通信，模式切换与消息传递的开销大、性能受损。Minix、QNX 是代表。
- 混合内核（Hybrid Kernel）：**折中方案，既保留微内核的模块化结构，又把性能敏感的服务（如图形、文件系统）放回内核态运行。Windows NT 系列、macOS（XNU）属于此类。

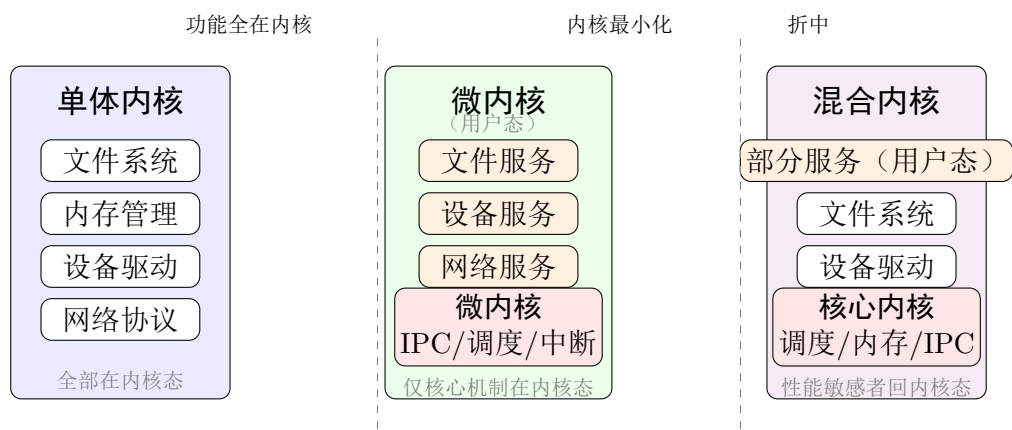


图 1.1: 三种内核结构的服务分布对比：单体内核（全在内核态）、微内核（服务下沉用户态）、混合内核（核心 + 部分内核态服务）

表 1.2: 三种内核结构的多维对比

结构	服务运行位置	优点	缺点	代表
单体内核	几乎全部在内核态	调用直接、性能高	耦合紧、可靠性差、难维护	Linux
微内核	仅核心机制在内核态	内核小、可靠、可扩展、易移植	频繁 IPC、性能损失	Minix, QNX
混合内核	核心 + 性能敏感服务在内核态	兼顾性能与模块化	结构复杂、边界模糊	Windows, macOS

1.3 用户态与内核态

硬件通过给 CPU 设定不同的**执行模式**来实现保护。至少需要两种模式：

- **用户态 (user mode)**：应用程序运行于此，只能访问自己的地址空间，不能执行可能危及系统的指令，也不能直接操纵设备。
- **内核态 (kernel mode，又称核心态/管理态)**：OS 内核运行于此，可访问全部内存与外设，可执行特权指令。

1.3.1 特权指令

特权指令 (privileged instruction) 是指那些只能在核心态执行、如果在用户态执行会触发陷阱 (trap) 的指令，例如：

- 开关中断 (cli/sti)、加载中断向量表；
- 修改页表基址寄存器、刷新 TLB；
- 直接进行 I/O 端口读写；
- 设置 CPU 模式位、停机指令 (halt)。

与之相对，加、减、跳转、访存等是**非特权指令**，两种模式下都能执行。核心态并非「性能更高」，它只是一种权限：CPU 通过一个模式位（存放在程序状态字/状态寄存器中）来判断当前是否允许执行特权指令。

1.3.2 模式切换与双模式的意义

用户态到内核态的切换由三类事件触发：**系统调用**（主动请求服务）、**中断**（外部设备异步发来）、**异常**（当前指令引发，如除零、缺页）。内核态返回用户态则通过一条专门的**特权返回指令**完成。

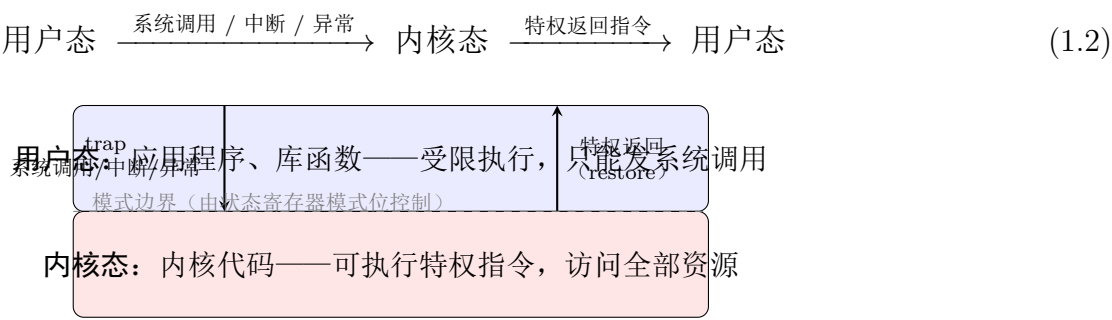


图 1.2: 双模式切换：用户态经 trap 陷入内核态，内核经特权返回指令回到用户态

为什么需要双模式？ 如果所有代码都能执行特权指令，那么一个程序的一条错误指令或一次恶意访问就可能覆盖内核数据、关闭中断或直接读写磁盘，从而使整个系统崩溃或失控。双模式把「可信的内核」与「不可信的应用」在权限上彻底隔开：应用想干特权的事，必须通过受控入口请求内核代劳，内核在入口处检查权限、参数与合法性后才执行。这正是「保护」职能的硬件基础。

1.4 系统调用

系统调用（system call）是操作系统提供给用户程序的、请求内核服务的受控入口，是「用户态」与「内核态」之间唯一的合法通道。它表面上像普通函数调用，实际却会跨越模式边界、进入内核执行特权操作。

1.4.1 陷入（trap）机制与参数传递

用户程序通常不直接书写系统调用指令，而是调用 C 库（如 glibc）提供的**包装函数**。包装函数把**系统调用号**放入约定寄存器，把参数按约定摆放，再执行一条能引发陷入的指令——在 x86 上是 `int 0x80` 或 `sysenter/syscall`，在 RISC-V 上是 `ecall`，在 ARM 上是 `svc`。该指令使 CPU 切换到内核态，并跳转到由**中断/异常向量表**指定的内核入口。

参数有三种常见传递方式：

- **寄存器传参**：参数少时把参数放进指定寄存器，最快，但寄存器数量有限；
- **栈传参**：把参数压入用户栈，内核按指针读取；
- **参数块传参**：把参数放在内存块中，将块地址放入寄存器传给内核，适合参数很多或参数长度可变（如 `execve`）的场合。

1.4.2 系统调用的类别

系统调用数量众多，按功能可分为五大类：

表 1.3: 系统调用的常见类别

类别	典型调用	作用
进程控制	<code>fork, exec, exit, wait</code>	创建、替换、终止、等待进程
文件管理	<code>open, read, write, close, lseek</code>	打开、读写、定位、关闭文件
设备管理	<code>ioctl, read, write</code>	请求设备操作与控制
信息维护	<code>getpid, time, alarm</code>	获取/设置系统与进程信息
进程通信	<code>pipe, socket, shmget, send/recv</code>	管道、共享内存、套接字通信

1.4.3 实例：read 的完整流程

下图为一次 read(fd, buf, n) 从用户态到内核态再返回的完整路径。左侧为运行模式，右侧为具体动作，中间虚线是模式边界：

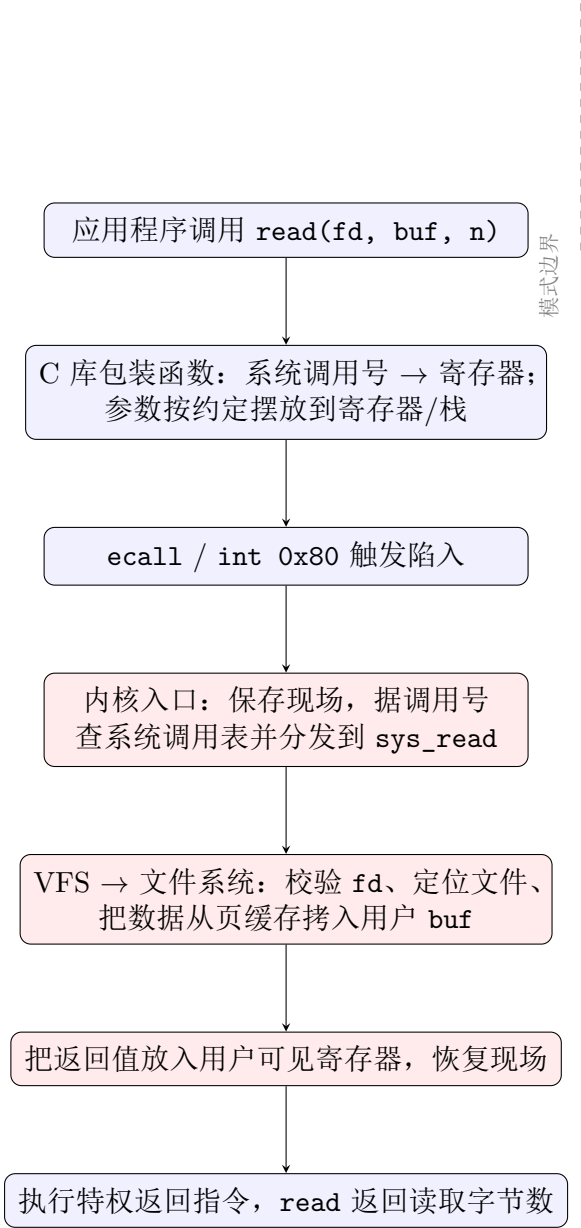


图 1.3: read 系统调用的完整路径：库包装 → 陷入 → 分发 → 服务 → 返回用户态

内核侧的通用分发逻辑可概括为下面的算法：

Algorithm 1 系统调用的陷入与分发（内核侧）

- 1: 硬件/入口保存用户态现场：PC、状态寄存器、通用寄存器入内核栈
- 2: 从约定寄存器读出系统调用号 `nr`
- 3: **if** `nr` 越界或无效 **then**
- 4: **return** 错误码（如 `-ENOSYS`）
- 5: **end if**
- 6: 按约定（寄存器/栈/参数块）取出参数，并做拷贝与合法性校验
- 7: 调用 `syscall_table[nr]` 指向的服务例程
- 8: 把服务例程的返回值写入用户可见寄存器
- 9: 恢复现场，执行特权返回指令，回到用户程序

由此可见，一次系统调用的开销 = 模式切换 + 现场保存/恢复 + 参数与结果拷贝 + 内核服务本身。它比普通函数调用昂贵得多，因此高性能程序常通过批量化（一次读写更多数据）或减少调用次数（缓冲、mmap）来摊薄这一开销。

1.5 中断、异常与陷阱

1.5.1 三者的区别

「中断」常被笼统地用来指代所有「打断当前执行、转入内核」的事件，但严格来说三者来源不同。关键区分点是**同步性**：它发生在当前指令流的哪个位置、是否可预期。

表 1.4: 中断、异常与陷阱的对比				
类型	来源	同步/异步	典型例子	处理后的返回点
中断 (Interrupt)	外部设备/时钟	异步	时钟、网卡、键盘	被中断指令的下一条指令
异常 (Exception)	当前指令执行	同步	除零、缺页、非法指令	修复后重执行（或终止进程）
陷阱 (Trap)	程序主动请求	同步	系统调用、断点	下一条指令

记忆要点：**中断**是「意外来访」（来自外部、任意时刻），**异常**是「自我报错」（由当前指令引起、可复现），**陷阱**是「主动敲门」（程序有意请求内核服务）。系统调用在实现上正是利用陷阱机制进入内核的。

1.5.2 中断处理流程

无论哪类事件，内核的处理骨架都是一致的，可概括为四步：**保存现场** → **查向量表** → **执行处理程序** → **恢复现场并返回**。

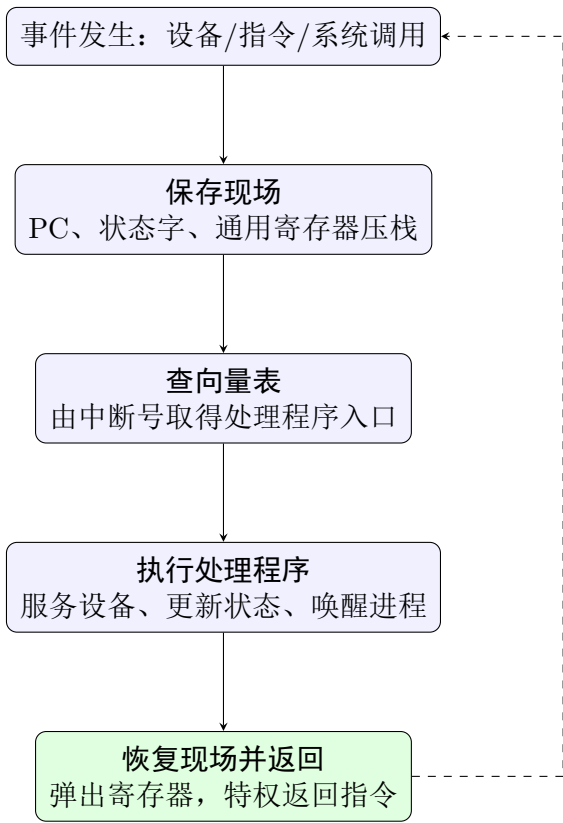


图 1.4: 中断处理流程：保存现场 → 查向量表 → 处理 → 恢复返回

中断向量表是一张由硬件索引的数组：每个中断号对应一个处理程序入口地址。事件发生时，硬件自动把**程序计数器**、**状态寄存器**等最小现场压栈（或保存到约定位置），再按中断号跳转；需要保存更多寄存器时由处理程序（软件）补齐。处理完毕后按相反顺序恢复，使被中断的程序像什么都没发生一样继续运行——这一「透明性」是中断机制最重要的性质。

1.5.3 时钟中断与抢占

时钟中断是操作系统夺回 CPU 控制权的手段。硬件定时器以固定周期（如每 1 ms）产生一次时钟中断；每次中断，内核都获得执行机会，可以：

- 更新时间、递减当前进程的时间片计数；
- 判断当前进程是否用完时间片，或是否有更高优先级进程就绪；
- 若是，则触发**抢占**——保存当前进程的上下文，切换到另一个进程。

没有时钟中断，一个陷入死循环的用户程序将永久占据 CPU，内核再也无法运行。因此，时钟中断是**抢占式调度**的物理基础，也是「分时」得以成立的前提。

1.6 上下文切换的开销

当内核决定从进程 A 切换到进程 B 时，必须保存 A 的上下文（寄存器、PC、栈指针、内存管理信息等），装入 B 的上下文。这一过程称为**上下文切换**。它的直接开销可分解为三段：

$$T_{\text{context}} = T_{\text{save}} + T_{\text{dispatch}} + T_{\text{restore}} \quad (1.3)$$

其中 T_{save} 为保存旧进程现场， T_{dispatch} 为调度器选择下一个进程， T_{restore} 为装入新进程现场。除直接开销外，还存在**间接开销**：切换后新进程的 TLB 与缓存大多失效，需要一段「冷启动」才能恢复命中率，这部分记为 T_{cold} ，往往远大于直接开销。若把时间片记为 q ，则上下文切换占用的 CPU 时间比例为

$$\eta = \frac{T_{\text{context}} + T_{\text{cold}}}{q + T_{\text{context}} + T_{\text{cold}}} \approx \frac{T_{\text{context}} + T_{\text{cold}}}{q}. \quad (1.4)$$

由上式可见：**时间片越小，切换越频繁，开销比例越高**；但时间片过大又会牺牲交互响应速度。调度器必须在「低开销」与「快响应」之间权衡。

表 1.5: 影响上下文切换开销的主要因素

因素	影响
需保存的寄存器数量	寄存器越多， T_{save} 、 T_{restore} 越大
是否切换地址空间	切换页表基址会牵动 TLB 刷新，间接开销显著上升
TLB / 缓存状态	切换后大量失效， T_{cold} 可达直接开销的数倍
调度器复杂度	选择算法的代价计入 T_{dispatch} （ $O(1)$ 与遍历有别）
CPU 与内存层次结构	流水线、多级缓存的深度影响现场恢复代价

例题：一次上下文切换开销估算

设某系统时间片 $q = 10 \text{ ms}$ ，一次切换中：保存现场 $T_{\text{save}} = 1.2 \mu\text{s}$ ，调度决策 $T_{\text{dispatch}} = 0.8 \mu\text{s}$ ，恢复现场 $T_{\text{restore}} = 1.0 \mu\text{s}$ ；切换后新进程因 TLB/缓存失效平均额外损失 $T_{\text{cold}} = 4.0 \mu\text{s}$ 。求切换的总开销及 CPU 有效利用率。

解：直接开销

$$T_{\text{context}} = 1.2 + 0.8 + 1.0 = 3.0 \mu\text{s}, \quad (1.5)$$

计入间接开销后，一次切换实际占用 CPU

$$T_{\text{total}} = T_{\text{context}} + T_{\text{cold}} = 3.0 + 4.0 = 7.0 \mu\text{s}. \quad (1.6)$$

每个时间片只发生一次切换，故有效利用率

$$\frac{q}{q + T_{\text{total}}} = \frac{10000}{10000 + 7} \approx 99.93\%, \quad (1.7)$$

即开销约 0.07%。若为提升响应速度把时间片减半为 1 ms ，则开销比例升至 $7/1000 = 0.70\%$ ——提高了十倍。可见时间片的选择直接影响系统效率，这正是「时间片大小」成为分时系统核心调参项的原因。

1.7 本章小结

- OS 兼具资源管理者、抽象提供者、控制程序三重身份；其形态随硬件与需求演化为批处理、分时、实时。
- 内核结构分单体内核、微内核、混合内核三类，本质是「性能」与「可靠性/模块化」之间的取舍。
- 硬件提供用户态/内核态双模式，特权指令只能在核心态执行，模式位是保护机制的基石。
- 系统调用是用户请求内核服务的受控入口，经 trap 陷入、分发、执行、返回，涉及寄存器/栈/参数块三种传参方式。
- 中断、异常、陷阱按同步性区分；中断处理遵循「保存现场 → 查向量表 → 处理 → 恢复返回」的骨架，时钟中断是抢占式调度的基础。
- 上下文切换有直接开销与缓存失效的间接开销，其占比随 T_{context}/q 上升，需与响应速度权衡。

1.8 常见误区

- 「内核态更快」：错误。核心态只是权限更高，指令执行速度与用户态相同；其「快」来自能直接操控硬件、免去模式切换，而非运算本身。
- 「系统调用就是普通函数调用」：错误。它必然跨越模式边界，包含现场保存、参数校验与结果拷贝，开销远大。
- 「中断一定是外设引起的」：不严谨。中断通常指异步外部事件，而系统调用走的是陷阱、除零走的是异常，三者来源不同。
- 「微内核一定更好」：不一定。微内核提升可靠性与可扩展性，但频繁 IPC 带来性能损失，需按场景取舍。
- 「上下文切换越少越省」就一定好：未必。时间片过大会拖慢交互响应，调度策略应同时考虑效率与响应。

第二章 进程与线程

进程是操作系统进行资源分配和调度的基本单位，线程是 CPU 调度的基本单位。本章从进程的状态模型出发，讲清 PCB、上下文切换与线程模型。

2.1 进程概念

操作系统要在单个 CPU 上同时支撑多个程序的运行，就必须为「正在执行的程序」建立一个统一的抽象与数据结构。这个抽象就是**进程 (process)**：它既描述了「一段正在运行的代码及其数据」，又是操作系统进行**资源分配与保护**的独立单元。本节先厘清程序与进程的区别，再说明进程由哪些部分构成，最后区分并发与并行这两个常被混淆的概念。

2.1.1 程序与进程

程序 (program) 是存放在磁盘上的静态文件，由指令和数据构成，本身不占用 CPU，也不会「运行」。**进程 (process)** 则是程序在某个数据集合上的一次执行过程，是动态的：它有开始、有状态、有结束，并随执行推进而不断变化。同一个程序可以被多次加载，产生多个相互独立的进程（例如同时打开两个文本编辑器）。

表 2.1: 程序与进程的区别

维度	程序	进程
存在形式	磁盘上的静态文件	内存中动态执行的实体
组成	指令 + 数据	代码段 + 数据段 + PCB
生命周期	长期保存，可反复使用	有创建、运行、结束的过程
对应关系	一个程序可对应多个进程	一个进程通常对应一个（或换入的）程序
资源占用	不占 CPU、不占运行时资源	占用 CPU、内存、文件等资源

关键结论：程序是**静态**的，进程是**动态**的；程序可长期保存，进程有生命周期；同一程序可对应多个进程，而一个进程在运行中还可以通过 `exec` 换成另一个程序。进程状态用 `state(p)` 表示，其生命周期可写作一个状态序列，详见后文。

2.1.2 进程的组成

进程在内存中的实体由三部分构成，缺一不可：

$$\text{进程} = \text{代码段} + \text{数据段} + \text{进程控制块 (PCB)} \tag{2.1}$$

- **代码段 (text)**：可执行的机器指令，通常只读，可被运行同一程序的多进程共享。
- **数据段 (data)**：全局/静态变量、堆与栈等可读写数据，是进程私有内容。
- **PCB**：操作系统为该进程建立的管理数据结构，记录其标识、状态与资源，是进程存在的**唯一标志**。

代码段与数据段合称进程的**地址空间**，PCB 则游离于地址空间之外、由内核统一管理。下图给出进程内存映像与 PCB 的关系。

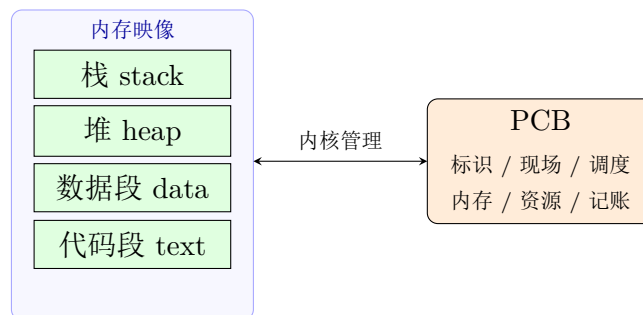


图 2.1: 进程的组成：代码段、数据段构成地址空间，PCB 由内核维护

2.1.3 并发与并行

这是两个极易混淆的概念，务必区分清楚。

表 2.2: 并发与并行的区别

维度	并发 (concurrency)	并行 (parallelism)
时刻关系	宏观同时、微观交替	同一时刻真正同时执行
硬件要求	单核即可实现	必须多核/多处理机
CPU 占用	任一时刻只有一个进程在占用 CPU	多个进程同时占用不同 CPU
典型效果	提高资源利用率、改善响应	提高单位时间完成的计算量

换言之，并发是「进程在时间上交错推进」的能力，并行是「进程真的同时跑」的能力。单核系统只能并发，多核系统才可能既并发又并行。

2.2 进程状态模型

进程在生命周期中并非一直在运行：它在等待 CPU、等待 I/O 或等待某个事件。操作系统用一组状态刻画进程此刻的处境，并规定状态之间哪些转换是合法的。状态模型是理解调度与阻塞的关键。

2.2.1 五种状态

- **新建 (new)**: 进程正在被创建，PCB 已分配但资源尚未完全就绪，尚未进入就绪队列。
- **就绪 (ready)**: 已获得除 CPU 之外的全部资源，只等被调度程序选中即可运行。
- **运行 (running)**: 已占有 CPU，正在执行指令（单核上至多一个进程处于此态）。
- **阻塞 (blocked)**: 因等待 I/O 或某事件而暂停，即使 CPU 空闲也不能运行。
- **终止 (terminated)**: 执行完毕或被撤销，等待系统回收其资源。

2.2.2 状态转换与触发事件

状态之间的每一次跳转都由一个明确的**触发事件**引起。合法转换与事件如下表。

表 2.3: 进程状态转换及其触发事件		
转换	触发事件	说明
新建 → 就绪	接纳 (admit)	资源就绪，插入就绪队列
就绪 → 运行	调度 (dispatch)	被调度程序选中并分派 CPU
运行 → 就绪	时间片到 / 被抢占	主动或被迫让出 CPU，重新排队
运行 → 阻塞	等待事件（如 I/O 请求）	进程主动放弃 CPU
阻塞 → 就绪	事件完成（如 I/O 完成）	回到就绪队列，仍不能直接运行
运行 → 终止	退出 (exit)	正常结束或被强制终止

关键约束：**阻塞态不能直接回到运行态**，必须先变为就绪态，再由调度程序决定何时运行；就绪到运行由调度触发，运行到就绪由时间片耗尽或被抢占触发。

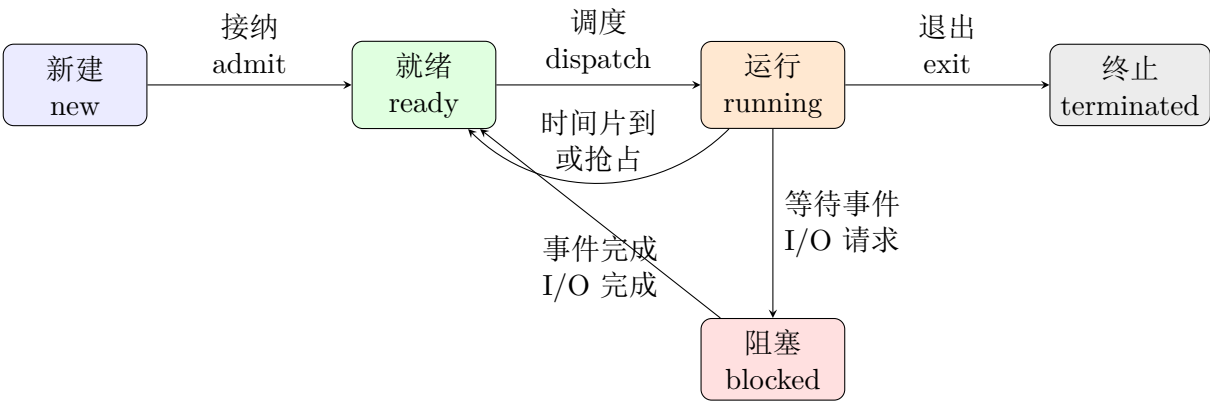


图 2.2: 进程五状态转换图：阻塞态必须先回到就绪态，不能直接进入运行态

2.2.3 状态转换的方程表示

用一个状态序列可以精确描述进程的一次完整经历。任意合法生命周期都可由如下「骨架」生成：

$$\text{new} \rightarrow \text{ready} \rightarrow \text{running} \rightarrow \{\text{ready}, \text{blocked}, \text{terminated}\} \tag{2.2}$$

$$\text{blocked} \rightarrow \text{ready} \rightarrow \text{running}, \quad \text{blocked} \not\rightarrow \text{running} \tag{2.3}$$

例如某进程 P 的一次执行可记为 $\text{new} \rightarrow \text{ready} \rightarrow \text{running} \rightarrow \text{blocked} \rightarrow \text{ready} \rightarrow \text{running} \rightarrow \text{ready} \rightarrow \text{running} \rightarrow \text{terminated}$ 。其中第二段 $\text{blocked} \rightarrow \text{ready}$ 由 I/O 完成触发，第三段 $\text{running} \rightarrow \text{ready}$ 由时间片耗尽触发。式 (3) 表明：若进程处于阻塞态，它**不可能**（用 $\not\rightarrow$ 表示）直接跳到运行态，忽略这一点是初学时的常见错误。

2.3 进程控制块 PCB

PCB 是进程存在的**唯一标志**：创建进程即建立 PCB，撤销进程即回收 PCB。操作系统通过 PCB 感知、控制和调度进程，因此 PCB 中必须完整记录「这个进程是谁、此刻在哪、拥有什么、该怎么调度」。

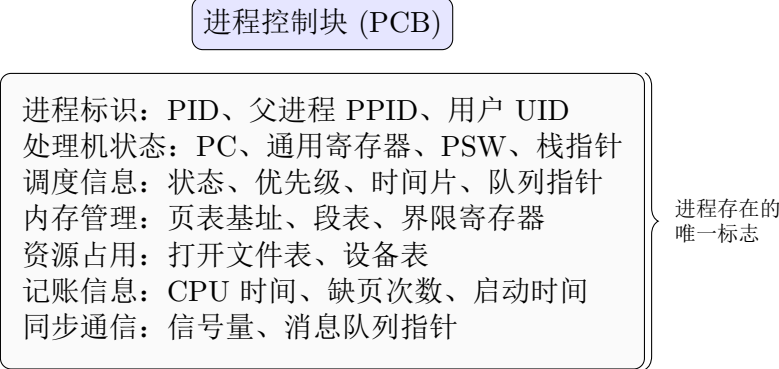


图 2.3: PCB 的典型字段组织：标识、现场、调度、内存、资源、记账、通信
各字段的用途逐项说明如下。

表 2.4: PCB 主要字段及其用途

分类	字段	用途
进程标识	PID、父进程 PPID、用户 UID	唯一标识进程及其归属与权限
处理机状态	通用寄存器、PC、PSW、栈指针	保存/恢复现场，支撑上下文切换
进程调度	状态、优先级、时间片、队列指针	提供调度决策所需的全部依据
内存管理	页表基址、段表、界限寄存器	描述地址空间并实施访问保护
I/O 与资源	打开文件表、设备表、信号量	记录进程占用的资源与通信对象
记账信息	CPU 时间、缺页次数、启动时间	用于统计、计费与性能分析

2.3.1 切换时的保存与恢复

当进程离开 CPU 时，内核把「处理器现场」写入其 PCB；当下次被调度时，再从 PCB 恢复到处理器寄存器。核心是以下寄存器集合：

现场 = { PC, 通用寄存器, PSW, 栈指针 }

(2.4)

其中 PC 决定「执行到哪条指令」，通用寄存器保存中间结果，PSW 保存标志位与特权级，栈指针决定函数调用与返回的位置。保存与恢复必须**成对且完整**，否则进程恢复后会出现结果错乱甚至崩溃。此外，PCB 中的状态、队列指针等字段也会被同步更新，以保证调度程序读到的是最新状态。

2.4 上下文切换

上下文切换 (context switch) 是内核保存当前进程现场、调入下一个进程现场的过程。它把「一个进程停下、另一个进程开始」在硬件层面真正实现。切换本身不产生任何用户价值，属于**纯开销**，因此其效率直接影响系统性能。

2.4.1 切换步骤

1. 保存当前进程 P_0 的 PC、通用寄存器、PSW、栈指针到 P_0 的 PCB。
2. 更新 P_0 的状态（运行 $\rightarrow$ 就绪，或运行 $\rightarrow$ 阻塞），并插入相应队列。
3. 调用调度程序，按策略从就绪队列中选择下一个待运行进程 P_1 。
4. 恢复 P_1 的 PCB 中保存的现场到处理器寄存器。
5. 切换地址空间（更新页表基址寄存器、刷新 TLB），返回用户态继续执行。

第 1、4 步是现场读写，第 3 步是策略决策，第 5 步是地址空间与高速缓存相关的隐性代价。

2.4.2 开销来源

- **寄存器保存/恢复**：读写 PCB 与寄存器组，需要若干条内核指令。
- **模式切换**：用户态与内核态之间的特权级往返。
- **地址空间与缓存**：切换页表、失效 TLB，并污染 cache，使新进程初期命中率下降。
- **调度开销**：遍历就绪队列、计算优先级的策略成本。

$$T_{\text{ctx}} = T_{\text{save}} + T_{\text{dispatch}} + T_{\text{restore}} \quad (2.5)$$

其中 T_{save} 、 T_{restore} 为寄存器与现场读写开销， T_{dispatch} 为调度选择开销。切换开销与硬件支持密切相关，通常在微秒量级。在时间片轮转中，若时间片为 q 、一次切换为 T_{ctx} ，则 CPU 的**有效利用率**与**切换开销占比**分别为

$$U = \frac{q}{q + T_{\text{ctx}}}, \quad \text{开销占比} = \frac{T_{\text{ctx}}}{q + T_{\text{ctx}}} \times 100\% \quad (2.6)$$

时间片越短，切换越频繁，开销占比越高；因此必须在响应时间与切换开销之间折中。

2.4.3 数值例题：上下文切换开销

设某系统采用时间片轮转调度，时间片 $q = 5\text{ ms}$ ，一次上下文切换平均耗时 $T_{\text{ctx}} = 0.2\text{ ms}$ 。试计算单个时间片的切换开销占比，并分析 q 变化的影响。

由式 (6)：

$$\text{开销占比} = \frac{0.2}{5 + 0.2} \times 100\% = \frac{0.2}{5.2} \times 100\% \approx 3.85\%$$
$$U = \frac{5}{5.2} \approx 96.15\%$$

即约 3.85% 的 CPU 时间被用于切换。若系统中同时有 4 个就绪进程，则一个完整轮转周期为 $n(q+T_{\text{ctx}}) = 4 \times 5.2 = 20.8\text{ ms}$ ，其中切换耗去 $4 \times 0.2 = 0.8\text{ ms}$ 。单位时间内发生的切换次数为 $N_{\text{sw}} = \frac{1000}{q + T_{\text{ctx}}} = \frac{1000}{5.2} \approx 192\text{ 次/秒}$ 。下表对比不同时间片下的开销占比。

表 2.5: 时间片 q 对切换开销的影响 ($T_{\text{ctx}} = 0.2\text{ ms}$)

时间片 q (ms)	开销占比	有效利用率 U	切换次数/秒
1	16.7%	83.3%	833
5	3.85%	96.15%	192
10	1.96%	98.04%	98
50	0.40%	99.60%	20

结论：把 q 从 5 ms 降到 1 ms，有效利用率从 96.15% 跌到 83.3%——时间片过短会让 CPU 忙于「换人」而非「干活」。反之 q 过大则响应变差，退化为先来先服务。

2.5 调度队列与调度器

操作系统用一组队列组织所有进程，用调度器在队列之间搬运进程。理解队列结构是理解调度算法的基础。

2.5.1 就绪队列与等待队列

- 就绪队列 (ready queue): 所有处于就绪态、等待 CPU 的进程，通常以链表或优先队列实现。
- 等待队列 (wait queue): 因等待 I/O 或某事件而阻塞的进程，通常按设备或事件类型各维护一个。

进程在队列之间的迁移可用下图表示：被调度选中者进入 CPU；等待 I/O 者进入等待队列；I/O 完成后回到就绪队列。

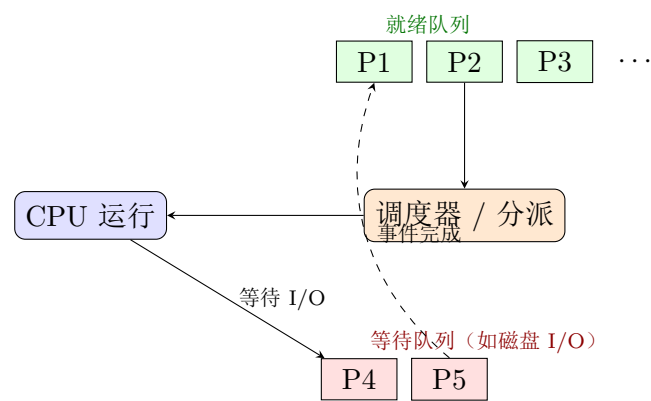


图 2.4: 就绪队列、CPU 与等待队列之间的进程迁移

2.5.2 调度器的层次

按作用范围与频率，调度器可分为三级。

表 2.6: 三级调度器		
调度器	职责	频率
长程调度（作业调度）	从后备作业中选择进入内存，决定多道程序度	最低
中程调度（交换调度）	决定进程换入/换出内存，缓解内存压力	中等
短程调度（CPU 调度）	从就绪队列中选择下一个占用 CPU 的进程	最高

短程调度器即通常所说的「CPU 调度器」，它每次上下文切换都可能被触发，直接决定交互响应与吞吐。多数系统并不实现完整的三级调度，但概念上区分它们有助于理解内存与 CPU 资源的耦合。

2.6 线程

进程既是资源分配单位又是执行单位，导致「若要并发，就必须为每个执行流单独分配地址空间」，代价高昂。**线程 (thread)** 把执行流从资源分配中解耦：同一进程内的多个线程共享地址空间与资源，却各自独立执行，从而以更小的开销获得更强的并发能力。线程是 **CPU 调度的基本单位**，进程是 **资源分配的基本单位**。

2.6.1 进程与线程的对比

表 2.7: 进程与线程对比		
维度	进程	线程
定位	资源分配单位	调度执行单位
地址空间	相互独立	同进程内共享
切换开销	大（含地址空间与 TLB 切换）	小（仅寄存器、栈与 PC）
通信方式	IPC、管道、共享内存	直接读写共享变量，快但需同步
隔离性	一个进程崩溃不影响其他进程	一个线程崩溃可能拖垮整个进程
拥有资源	文件、内存、设备	基本不拥有，借用所属进程资源
私有内容	整个地址空间	栈、寄存器、PC、线程局部存储

2.6.2 线程的地址空间

下图把「共享」与「私有」两部分画在一起：同一进程的多个线程共享代码段、数据段与打开文件表，但各自拥有私有的栈、寄存器和 PC。

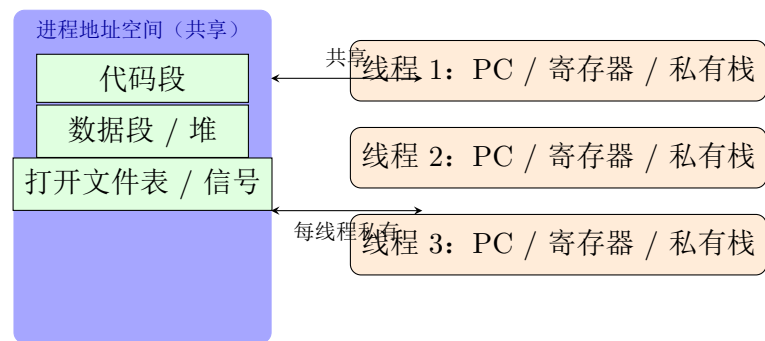


图 2.5: 线程的地址空间：代码与数据共享，栈、寄存器、PC 每线程私有

2.6.3 用户级线程与内核级线程

线程由谁管理，决定了它的可见性与并发能力。

表 2.8: 用户级线程 (ULT) 与内核级线程 (KLT)		
维度	用户级线程 ULT	内核级线程 KLT
管理者	用户态线程库	操作系统内核
内核是否可见	不可见	可见，作为独立调度对象
切换开销	很小，无需陷入内核	较大，需陷入内核
多核并行	不能（一个 KLT 承载全部 ULT）	可以，多个 KLT 可并行运行
阻塞影响	一个线程阻塞导致整个进程阻塞	单个线程阻塞不影响其他线程
典型实现	早期 pthread 的 N:1	Linux 内核线程、NPTL

2.6.4 多线程模型

用户级线程与内核级线程之间的映射方式即多线程模型。

表 2.9: 多线程映射模型

模型	映射关系	特点
多对一 (many-to-one)	多 ULT $\rightarrow$ 1 KLT	切换快，但一线程阻塞即全进程阻塞，无法并行
一对一 (one-to-one)	1 ULT $\rightarrow$ 1 KLT	并行好、阻塞隔离，但线程创建与切换开销大
多对多 (many-to-many)	多 ULT $\rightarrow$ 多 KLT	折中，允许受限并行，实现复杂

一对一模型（如 Linux NPTL）实现简单、并行性好，但线程数受内核限制；多对多模型试图在灵活性与开销之间取得平衡，但内核与线程库的协作复杂，是实践中较少采用的折中方案。

2.7 进程间通信简介

进程之间相互独立，若要让它们交换数据，就需要进程间通信 (IPC)。主流方式分为两大类：共享内存与消息传递。前者在地址空间中划出一块共享区域，进程直接读写，速度快但必须由用户自行同步；后者通过内核在进程之间搬移消息，天然同步、易于跨机器，但每次通信都要陷入内核。二者对比如下表。

表 2.10: 共享内存与消息传递

维度	共享内存	消息传递
通信机制	映射同一物理内存，直接读写	以内核为中介收发消息
速度	快（建立后无需系统调用）	较慢（每次通信需陷入内核）
同步	用户负责，需信号量/互斥锁	发送/接收本身提供同步语义
隔离性	弱，共享区出错会相互影响	强，仅通过消息交互
是否跨主机	否	是（如 socket）
典型接口	shmget/mmap	pipe、msgqueue、socket

选择原则：追求性能且进程同机时用共享内存，并配合同步机制；追求简单与安全、或需要跨主机通信时用消息传递。同步与死锁的细节将在后续章节展开。

2.8 本章小结

- 进程是程序的一次执行，由代码段、数据段和 PCB 组成；程序静态、进程动态。
- 进程有新建、就绪、运行、阻塞、终止五种状态；阻塞态必须先回到就绪态才能运行。
- PCB 是进程存在的唯一标志，保存标识、现场、调度、内存与资源等全部信息。
- 上下文切换由「保存 $P_0 \rightarrow$ 调度 $\rightarrow$ 载入 P_1 」构成，是纯开销；开销占比 = $T_{ctx}/(q + T_{ctx})$ 。
- 就绪队列与等待队列配合三级调度器，完成进程在 CPU、内存与 I/O 之间的迁移。
- 线程是调度基本单位，进程是资源分配单位；同一进程的线程共享地址空间，私有栈与寄存器。
- 线程模型有多对一、一对一、多对多；用户级线程切换快但不能并行，内核级线程可并行但开销大。
- 进程间通信分共享内存与消息传递：前者快而需同步，后者安全且可跨主机。

2.9 常见误区

- **把程序当进程：**程序是静态文件，进程是动态执行体；同一程序可对应多个进程。
- **认为阻塞可以直接变运行：**阻塞只回到就绪，必须再经调度才可能运行。
- **混淆就绪与阻塞：**就绪是「只差 CPU」，阻塞是「在等事件」，前者可被调度，后者不能。
- **把并发当并行：**单核只能并发；并行需要多核，二者不是同义词。
- **认为线程切换与进程切换等价：**线程切换不换地址空间，开销小得多。
- **认为多线程一定更快：**线程间共享数据需要同步，竞争激烈时并行收益可能被锁开销抵消。
- **认为用户级线程能利用多核：**一个内核线程承载全部用户级线程时，无法真正并行。
- **低估上下文切换成本：**时间片过短会使切换成为主要开销，CPU 有效利用率显著下降。

第三章 CPU 调度

CPU 调度决定「就绪队列中的哪个进程、在什么时刻、占用 CPU 多久」。本章先用周转/等待/响应/吞吐四类指标统一度量，再依次讲清 FCFS、SJF/SRTF、时间片轮转 RR、优先级与多级反馈队列 MLFQ 的机制与取舍。

3.1 调度指标

调度算法的优劣需要统一度量。设进程 P_i 的到达时间为 a_i 、运行（CPU 突发）时间为 b_i 、完成时间为 c_i ：

$$T_{\text{周转}} = c_i - a_i \quad (3.1)$$

$$T_{\text{等待}} = T_{\text{周转}} - b_i = c_i - a_i - b_i \quad (3.2)$$

$$T_{\text{响应}} = T_{\text{首次运行}} - a_i \quad (3.3)$$

$$\text{吞吐量} = \frac{\text{单位时间内完成的作业数}}{\text{时间}} = \frac{n}{T_{\text{总}}} \quad (3.4)$$

其中周转时间衡量「从提交到完成」的总时长；等待时间衡量进程在就绪队列中「干等」的时长；响应时间衡量「从提交到第一次得到响应」的时长——对交互式系统尤为重要；吞吐量衡量单位时间的完成能力。通常用平均值比较算法： $\overline{T_{\text{周转}}} = \frac{1}{n} \sum_i (c_i - a_i)$ ， $\overline{T_{\text{等待}}} = \frac{1}{n} \sum_i (c_i - a_i - b_i)$ 。

表 3.1: 四类调度指标

指标	定义	关注点
周转时间	完成时刻 - 到达时刻	批处理、整体效率
等待时间	周转时间 - 运行时间	就绪队列排队的代价
响应时间	首次运行时刻 - 到达时刻	交互式、用户体验
吞吐量	单位时间完成的作业数	系统整体产能

注意：等待时间与响应时间不同——抢占式算法中进程可能被多次打断，响应时间只关心「第一次」得到 CPU 的时刻。CPU 利用率与吞吐量都希望减少 上下文切换开销：切换本身不产生用户价值。

3.2 先来先服务 FCFS 与最短作业优先 SJF

考虑如下进程集合（本章统一使用）：

表 3.2: 示例进程集合

进程	到达时间 a_i	运行时间 b_i	优先级
P_1	0	7	3
P_2	2	4	1
P_3	4	1	4
P_4	5	4	2

3.2.1 先来先服务 FCFS

FCFS 按到达顺序排队，**非抢占**：一旦 CPU 分配给某进程，就运行到完成或主动阻塞。实现最简单，但存在 **护航效应 (convoy effect)**：一个长作业会拖住后面所有短作业。

FCFS 调度序列： $P_1[0, 7] \rightarrow P_2[7, 11] \rightarrow P_3[11, 12] \rightarrow P_4[12, 16]$ 。完成时刻 $c = (7, 11, 12, 16)$ ，因此

$$\overline{T_{\text{周转}}} = \frac{7 + 9 + 8 + 11}{4} = 8.75, \quad \overline{T_{\text{等待}}} = \frac{0 + 5 + 7 + 7}{4} = 4.75 \quad (3.5)$$

3.2.2 最短作业优先 SJF

SJF 非抢占，每次选择运行时间最短的就绪进程。在「所有进程同时到达」的理想假设下，SJF 可证明平均等待时间最小（最优）。实际中运行时间只能预测（如指数平均），且长作业可能饿死。

按到达时刻推进：t = 0 只有 P₁，只能先跑；t = 7 时 P₂, P₃, P₄ 均就绪，选最短的 P₃；之后选 P₂，最后 P₄。序列为 P₁[0, 7] → P₃[7, 8] → P₂[8, 12] → P₄[12, 16]：

$$\overline{T}_{\text{周转}} = \frac{7 + 10 + 4 + 11}{4} = 8.00, \quad \overline{T}_{\text{等待}} = \frac{0 + 6 + 3 + 7}{4} = 4.00$$

(3.6)

3.2.3 最短剩余时间优先 SRTF

SRTF 是 SJF 的抢占版本：每当新进程到达，就比较「当前进程剩余时间」与「新进程运行时间」，若新进程更短则立即抢占。序列为 P₁[0, 2] → P₂[2, 4] → P₃[4, 5] → P₂[5, 7] → P₄[7, 11] → P₁[11, 16]：

$$\overline{T}_{\text{周转}} = \frac{16 + 5 + 1 + 6}{4} = 7.00, \quad \overline{T}_{\text{等待}} = \frac{9 + 1 + 0 + 2}{4} = 3.00$$

(3.7)

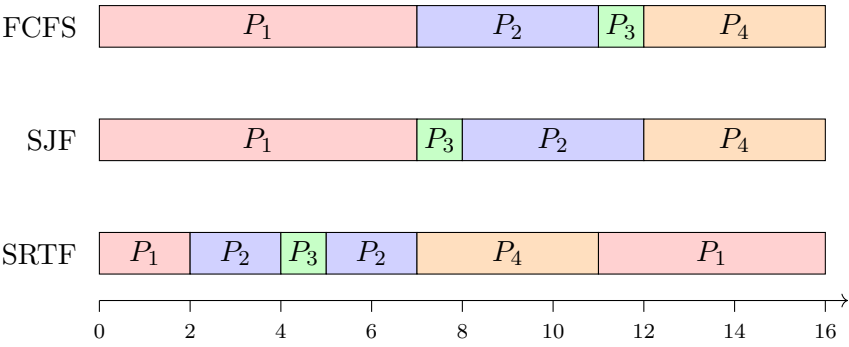


图 3.1: FCFS / SJF / SRTF 甘特图（同一进程集合，时间单位）

表 3.3: 三种非交互算法对比			
算法	抢占	平均周转	平均等待
FCFS	否	8.75	4.75
SJF	否	8.00	4.00
SRTF	是	7.00	3.00

SRTF 的指标最好，但抢占带来更多上下文切换；且三者都以「长作业饥饿」为代价换取短作业的响应。SJF/SRTF 的理论最优性依赖运行时间可预知，实际系统只能用历史行为估计。

3.3 时间片轮转 RR 与优先级调度

3.3.1 时间片轮转 RR

RR 为每个进程分配一个固定时间片 q ，就绪队列按 **FIFO 轮转**：进程用完一个时间片即被抢占并排到队尾；若在时间片内完成或主动阻塞则提前让出。RR **抢占**且天然公平，是分时系统的基石。

取与上表相同运行时间但**同时到达**的进程， $q = 2$ ：序列为

$$P_1[0, 2] \rightarrow P_2[2, 4] \rightarrow P_3[4, 5] \rightarrow P_4[5, 7] \rightarrow P_1[7, 9] \rightarrow P_2[9, 11] \rightarrow P_4[11, 13] \rightarrow P_1[13, 16]$$

完成时刻 $c = (16, 11, 5, 13)$ ，于是

$$\overline{T}_{\text{周转}} = \frac{16 + 11 + 5 + 13}{4} = 11.25, \quad \overline{T}_{\text{等待}} = \frac{9 + 7 + 4 + 9}{4} = 7.25, \quad \overline{T}_{\text{响应}} = \frac{0 + 2 + 4 + 5}{4} = 2.75 \quad (3.8)$$

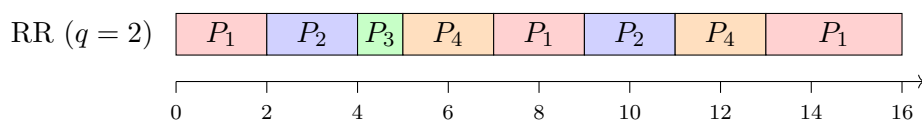


图 3.2: RR 甘特图（时间片 $q = 2$ ，进程同时到达）

3.3.2 时间片大小的影响

时间片 q 是 RR 最重要的参数，直接决定「响应性」与「切换开销」的折中：

- $q \rightarrow \infty$ ：RR 退化为 FCFS，响应时间变差，但切换最少。
- $q \rightarrow 0$ ：每个指令都切换，上下文切换开销吞掉有效计算，吞吐量骤降。
- q 应略大于「一次上下文切换时间」且覆盖绝大多数 CPU 突发，经验上取 $10 \sim 100$ ms。

对同一集合（同时到达）计算不同 q （切换开销忽略）：

表 3.4: 时间片 q 对指标的影响（同时到达）			
时间片 q	平均等待	平均响应	平均周转
1	7.00	1.50	11.00
2	7.25	2.75	11.25
4	7.50	5.25	11.50
∞ (FCFS)	7.50	7.50	11.50

结论：在上例中总完成时间恒为 16（单 CPU、不空闲），但**时间片越小，响应越快**（首轮轮转更早到达每个进程），代价是切换更频繁；时间片越大越接近 FCFS。因此 q 的选取要在响应时间与吞吐量之间折中。

3.3.3 优先级调度

每个进程赋予一个优先级，调度器总是选择**优先级最高**的就绪进程。约定数值越小优先级越高。优先级可**静态**给定，也可**动态**调整。主要问题有二：

- **饥饿 (starvation)**：低优先级进程可能长期得不到 CPU。
- **优先级反转 (priority inversion)**：低优先级进程持有高优先级进程所需的锁，导致后者被中优先级进程阻塞，可用优先级继承缓解。

对同时到达、优先级为 $P_2(1) > P_4(2) > P_1(3) > P_3(4)$ 的集合做非抢占优先级调度，序列为

$$P_2[0, 4] \rightarrow P_4[4, 8] \rightarrow P_1[8, 15] \rightarrow P_3[15, 16]$$
$$\overline{T}_{\text{周转}} = \frac{15 + 4 + 16 + 8}{4} = 10.75, \quad \overline{T}_{\text{等待}} = \frac{8 + 0 + 15 + 4}{4} = 6.75 \tag{3.9}$$

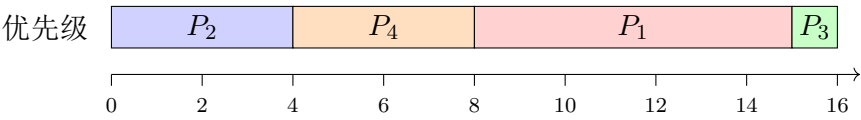


图 3.3: 非抢占优先级调度甘特图（数值小者优先）

RR 与优先级的结合即**优先级 + 时间片**：高优先级队列内轮转，既保响应又防独占。

3.4 多级反馈队列 MLFQ

MLFQ 不要求预知运行时间，而是用「多个优先级队列 + 反馈」动态推测进程类型（CPU 密集型还是 I/O 密集型）。核心规则：

1. **多队列、优先级递减**：设 $Q_0, Q_1, \dots, Q_{n-1}$ ， Q_0 优先级最高。总是调度最高优先级非空队列中的进程。
2. **时间片随层级递增**：高优先级队列用短时间片（如 Q_0 为 2），低优先级队列用长时间片（如 Q_1 为 4， Q_2 为 FCFS），以适配不同进程。
3. **新进程入最高队列**：所有新进程先进入 Q_0 ，享受最短响应。
4. **用完整个时间片则降级**：进程若耗尽分配的整个时间片仍未完成，说明偏 CPU 密集，被降到下一级队列。
5. **主动让出则保持/升级**：进程在时间片用完前主动让出（如等待 I/O），说明偏交互式，留在原队列（或提升），从而持续获得好响应。
6. **周期提升 (priority boost)**：每隔固定时间 S ，把所有进程重新提到 Q_0 ，防止长作业饥饿，并校正对 CPU/IO 类型的误判。
7. **同队列内轮转**：同一优先级队列内部按 RR 调度。

表 3.5: MLFQ 队列参数（示例）

队列	优先级	时间片	典型对象
Q_0	最高	2	新进程、交互式 (I/O 密集)
Q_1	中	4	短批处理
Q_2	最低	FCFS	CPU 密集型长作业

MLFQ 用「降级惩罚 CPU 密集、保留奖励 I/O 密集、周期提升兜底饥饿」三条机制，在无需预知运行时间的前提下同时兼顾响应时间与吞吐量，是通用操作系统最常用的调度框架之一。

第四章 同步与并发

多个进程或线程并发访问共享资源时会产生竞态，同步机制用于保证结果的可再现与正确性。本章从临界区问题的三个必要条件出发，依次介绍 Peterson 算法、信号量（P/V）与生产者-消费者、读者-写者问题、管程与条件变量，最后讨论死锁的四个必要条件、资源分配图与银行家算法。

4.1 临界区问题

多个进程并发访问共享数据，执行的相对顺序不确定，其最终结果依赖于调度时序，这种现象称为**竞态条件 (race condition)**。把访问共享资源的代码称为**临界区 (critical section)**。

4.1.1 三个必要条件

任何正确的临界区解决方案必须同时满足以下三个条件。

表 4.1: 临界区问题的三个必要条件

条件	含义	反例
互斥 (Mutual Exclusion)	任一时刻至多一个进程处于临界区	两进程同时写同一变量
前进 (Progress)	临界区空闲且有进程想进入时，能及时选出进入者，不被无关进程无限阻塞	只剩空闲却无人能进
有限等待 (Bounded Waiting)	一个进程发出进入请求后，在它之前允许其他进程进入的次数有上界	某进程被无限插队饿死

$$\text{互斥} \wedge \text{前进} \wedge \text{有限等待} \implies \text{正确同步} \tag{4.1}$$

仅靠「标志数组」或「轮转」单独使用都无法同时满足三条件：纯标志会互相礼让导致死锁，纯轮转虽保证互斥却违反前进。

4.1.2 Peterson 算法

Peterson 算法是满足三条件的经典软件方案，适用于两个进程。它组合了「意愿标志 flag」与「轮转变量 turn」，其中 turn 用于打破对称僵局。

Listing 4.1: Peterson 算法（进程 i ，对方为 $j = 1 - i$ ）

```
1 flag[i] = true;           // 声明我想进入临界区
2 turn = j;                 // 但先把机会让给对方
3 while (flag[j] && turn == j) { /* 忙等待 */ }
4
5 /* ----- 临界区 ----- */
6
7 flag[i] = false;         // 退出后撤销意愿
8 /* ----- 剩余区 ----- */
```

- 置 `flag[i]=true`，表明进程 i 有进入意图。
- 置 `turn=j`，把优先权交给对方。当两者同时竞争时，`turn` 的最后一次写入决定了谁等待。
- 循环等待：只要对方也想进入且 `turn` 指向对方，就让其先进入。
- 进入临界区，执行完后置 `flag[i]=false`，允许对方进入。

正确性：若两进程同时进入 while，则 turn 只能取一个值，其中一方必然满足 turn != j 而跳出，保证**互斥**；退出者一旦置 flag=false，等待者即可进入，满足**前进**；对方至多进入一次后 turn 被改写，满足**有限等待**。但 Peterson 依赖顺序一致的内存模型，在多核弱一致内存上需内存屏障。

4.2 信号量

4.2.1 P/V 操作

信号量 S 是一个受保护的整型变量，只能通过两个原子操作访问：

Listing 4.2: 信号量的 wait (P) 与 signal (V)

```
1 wait(S):           // P 操作 / 申请资源，可能阻塞
2     S.value--;      // 也可写为 while (S.value <= 0) ;
3     if (S.value < 0)
4         block();    // 无资源则入等待队列
5
6 signal(S):         // V 操作 / 释放资源，可唤醒等待者
7     S.value++;
8     if (S.value <= 0)
9         wakeup();   // 尚有等待者，唤醒其一
```

- **二元信号量 (binary semaphore):** 取值仅 {0,1}，退化为互斥锁，用于临界区互斥。
- **计数信号量 (counting semaphore):** 取值为非负整数 n ，表示可用资源数量，初值即资源总数。

P、V 必须成对出现；P 阻塞调用者，V 唤醒一个等待者。若把 P/V 原子性破坏，互斥就会失效。

4.2.2 生产者-消费者问题

设缓冲区容量为 n ，信号量 mutex 保护缓冲区本身的互斥访问，empty、full 分别表示空槽与满槽数量。

Listing 4.3: 生产者-消费者（有限缓冲区）

```
1 semaphore mutex = 1;    // 缓冲区互斥
2 semaphore empty = n;    // 空槽数
3 semaphore full  = 0;    // 满槽数
4
5 producer():             consumer():
6     while (true) {      while (true) {
7         item = produce();    wait(full);    // 等待有产品
8         wait(empty);        wait(mutex);    // 进入临界区
9         wait(mutex);        item = take();   // 取走并腾出
10        put(item);          signal(mutex);
11        signal(mutex);      signal(empty); // 空槽 +1
12        signal(full);       consume(item);
13    }                      }
```

顺序要点：必须先 wait(empty) 再 wait(mutex)。若颠倒，生产者可能持有 mutex 却在 empty 上阻塞，消费者无法进入而永久等待，形成死锁。

4.2.3 读者-写者问题

允许多个读者同时读，但写者必须独占。第一类（读者优先）可能使写者饥饿；第二类（写者优先）可能使读者饥饿。

Listing 4.4: 读者-写者（读者优先）

```
1 semaphore rw_mutex = 1; // 读写的互斥锁
2 semaphore mutex    = 1; // 保护 read_count
3 int read_count = 0;
4
5 reader():                               writer():
6     wait(mutex);                         wait(rw_mutex);
7     read_count++;                        /* 写操作 */
8     if (read_count == 1) wait(rw_mutex); signal(rw_mutex);
9     signal(mutex);
10    /* 读操作 */
11    wait(mutex);
12    read_count--;
13    if (read_count == 0) signal(rw_mutex);
14    signal(mutex);
```

4.3 管程与条件变量

管程 (monitor) 是把共享变量和操作它们的过程封装在一起的高级同步抽象：任一时刻只允许一个进程在管程内活动，互斥由编译器/运行库隐式保证，无需手工加锁。管程内用条件变量实现等待与唤醒。

表 4.2: 管程内条件变量的操作

操作	语义	说明
c.wait()	释放管程锁并阻塞于条件 c	被唤醒后须重新获得管程锁才返回
c.signal()	唤醒一个等待 c 的进程	若无等待者则该调用无效果

Listing 4.5: 管程实现生产者-消费者

```
1 monitor BoundedBuffer {
2     item buf[n]; int count = 0;
3     condition notFull, notEmpty; // 条件变量
4
5     void produce(item x) {
6         if (count == n) notFull.wait(); // 满则等待
7         buf[in] = x; in = (in + 1) % n; count++;
8         notEmpty.signal();             // 唤醒等待的消费者
9     }
```

```
10 item consume() {
11     if (count == 0) notEmpty.wait(); // 空则等待
12     item x = buf[out]; out = (out + 1) % n; count--;
13     notFull.signal();                // 唤醒等待的生产者
14     return x;
15 }
16 }
```

条件变量与信号量不同：信号量有计数值并可跨进程传递状态，而 `signal` 只在已有等待者时才生效，且不保存「许可」。因此需用 `while` 而非 `if` 重新检查条件（Mesa 语义下可能虚假唤醒）。

4.4 死锁

4.4.1 四个必要条件

死锁 (deadlock) 指一组进程因互相等待对方持有的资源而永久阻塞。四个条件必须同时成立才可能死锁；破坏任一条件即可预防。

表 4.3: 死锁的四个必要条件		
条件	含义	破坏策略
互斥 (Mutual Exclusion)	资源一次只能被一个进程占用	尽量共享（如只读文件）
占有并等待 (Hold and Wait)	持有资源的同时等待新资源	一次性申请全部资源
不可抢占 (No Preemption)	资源只能被持有者主动释放	允许抢占、回滚
循环等待 (Circular Wait)	存在进程-资源的环形等待链	给资源编号，按序申请

4.4.2 资源分配图

资源分配图用有向图描述时刻状态：圆形表示进程 P ，方形表示资源 R （内部黑点表示实例数）。 $P \rightarrow R$ 为请求边， $R \rightarrow P$ 为分配边。图中出现环是死锁的必要条件：单实例时环即死锁；多实例时环只是可能死锁，还需结合可用实例判断。

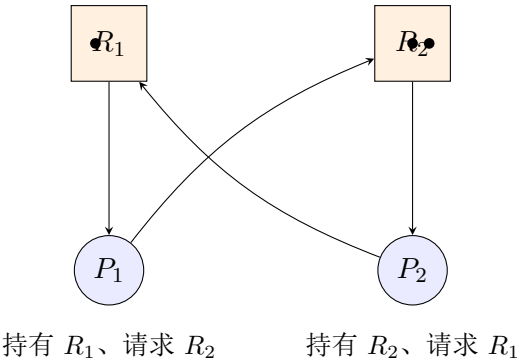


图 4.1: 资源分配图： P_1 与 P_2 互相请求对方持有的资源，形成环

4.4.3 银行家算法

银行家算法在进程提出资源请求时进行**安全性检查**：若分配后系统仍处于安全状态则满足请求，否则推迟，从而避免死锁。

$$Need[i] = Max[i] - Allocation[i] \quad (4.2)$$

$$Available' = Available + Allocation[i] \quad (\text{当 } Need[i] \leq Available) \quad (4.3)$$

Algorithm 2 银行家算法：安全性检查

```
1: 令  $Work = Available$ ,  $Finish[i] = \text{false}$  ( $i = 1, \dots, n$ )
2: while 存在  $i$  使  $Finish[i] = \text{false}$  且  $Need[i] \leq Work$  do
3:    $Work \leftarrow Work + Allocation[i]$ 
4:    $Finish[i] \leftarrow \text{true}$ 
5:   输出  $i$  (加入安全序列)
6: end while
7: if 所有  $Finish[i] = \text{true}$  then
8:   return 安全, 安全序列即输出顺序
9: else
10:  return 不安全 (可能死锁)
11: end if
```

以经典例子 $Available = (3, 3, 2)$ 说明：

表 4.4: 银行家算法示例 (资源类型 A, B, C)

进程	Max	$Allocation$	$Need$	完成顺序
P_0	(7,5,3)	(0,1,0)	(7,4,3)	4
P_1	(3,2,2)	(2,0,0)	(1,2,2)	1
P_2	(9,0,2)	(3,0,2)	(6,0,0)	5
P_3	(2,2,2)	(2,1,1)	(0,1,1)	2
P_4	(4,3,3)	(0,0,2)	(4,3,1)	3

检查过程： $Need[P_1] = (1, 2, 2) \leq (3, 3, 2)$ ，完成后 $Work = (5, 3, 2)$ ； $Need[P_3] = (0, 1, 1) \leq Work$ ， $Work = (7, 4, 3)$ ； $Need[P_4] = (4, 3, 1) \leq Work$ ， $Work = (7, 4, 5)$ ； $Need[P_0] = (7, 4, 3) \leq Work$ ， $Work = (7, 5, 5)$ ；最后 $Need[P_2] = (6, 0, 0) \leq Work$ 完成。

$$\text{安全序列: } P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2 \quad (4.4)$$

存在安全序列说明系统当前安全，即使各进程按此顺序依次申请，也能全部完成；否则系统进入不安全状态，应采用资源预分配、避免或恢复（抢占、回滚、终止进程）等策略。

第五章 内存管理

内存管理解决「多个程序如何共享有限物理内存」的问题：通过逻辑地址与物理地址的分离、重定位与保护机制，为每个进程提供独立地址空间；再以分页、分段及其组合把连续的虚拟地址映射到离散的物理帧，并用 TLB 与多级页表控制平均访存开销。

5.1 地址空间与重定位

5.1.1 逻辑地址与物理地址

- 逻辑地址（虚拟地址）：CPU 生成的地址，面向进程视角，从 0 开始连续编号。
- 物理地址：内存单元的真实地址，由内存总线送入存储体。
- 二者关系由内存管理单元在运行时完成转换；编译期只能确定逻辑地址。

地址绑定可在三个时机发生：

表 5.1: 地址绑定的时机

时机	说明	特点
编译时	生成绝对地址代码	程序位置固定，重定位需重新编译
装入时	装入时把逻辑地址改写为物理地址	一旦装入不能移动
执行时	运行时由 MMU 动态重定位	灵活，支持换入换出与虚拟内存

5.1.2 MMU 与基址/界限寄存器

内存管理单元（MMU）是 CPU 与内存之间的硬件，负责把逻辑地址翻译成物理地址并实施保护。最朴素的重定位方案为每个进程维护一对寄存器：

- 基址寄存器 (base)：进程可用物理内存的起始地址。
- 界限寄存器 (limit)：进程地址空间的长度。

$$\text{物理地址} = \text{base} + \text{逻辑地址}, \quad \text{合法} \iff 0 \leq \text{逻辑地址} < \text{limit} \tag{5.1}$$

若逻辑地址越过 limit，则触发越界陷阱（trap），交由内核处理，从而实现进程间的存储保护。

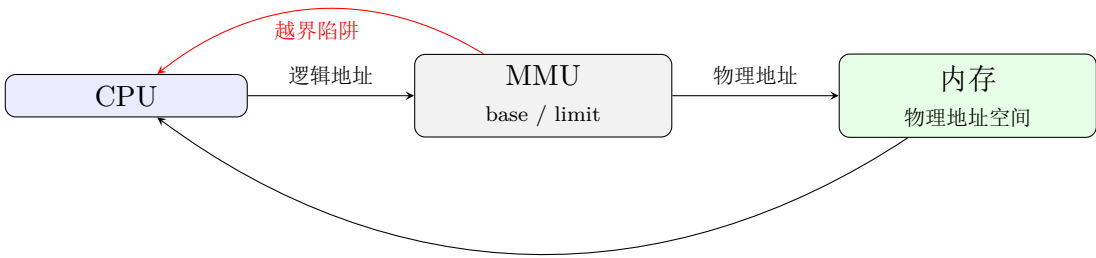


图 5.1: MMU 在逻辑地址与物理地址之间完成重定位

5.2 分页

5.2.1 页、帧与页表

分页把逻辑地址空间切成大小相等的**页 (page)**，把物理内存切成同样大小的**帧 (frame)**。页与帧大小相同且为 2 的幂（如 4KB），页可以装入任意空闲帧，从而消除外部碎片。

页表（page table）为每个进程建立「页号 → 帧号」的映射，通常存放在内存中。

$$\text{逻辑地址} = \underbrace{p}_{\text{页号}} \parallel \underbrace{d}_{\text{页内偏移}}, \quad p = \left\lfloor \frac{LA}{S} \right\rfloor, \quad d = LA \bmod S \quad (5.2)$$

$$PA = f \times S + d, \quad f = \text{页表}[p] \quad (5.3)$$

其中 S 为页大小， f 为页号 p 对应的帧号。由于 S 是 2 的幂，除以 S 等价于右移 $\log_2 S$ 位，取模等价于取低 $\log_2 S$ 位，因此地址翻译可完全由位切分实现。

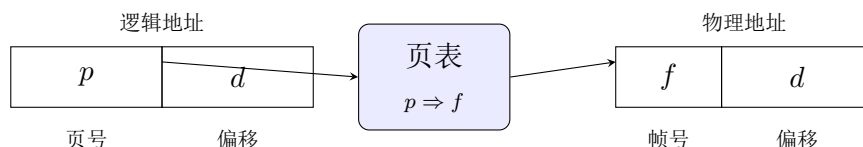


图 5.2: 分页地址翻译：页号查页表得帧号，偏移原样保留

5.2.2 页表项 PTE

页表项（Page Table Entry）除帧号外还记录若干控制位：

表 5.2: 页表项的主要字段

字段	位宽	作用
帧号 (frame number)	视物理内存大小而定	该页映射到的物理帧
有效位 (valid / present)	1	页面是否已装入内存，0 触发缺页
修改位 (dirty)	1	页面是否被写过，决定换出时是否回写
访问位 (reference)	1	是否被访问，供置换算法参考
保护位 (protection)	2-3	读/写/执行权限，越权触发保护异常

在 32 位、4KB 页的单级页表下，需要 2^{20} 个表项、约 4MB，且每个进程一份。为压缩开销引入**多级页表**：只把用到的下级页表装入内存，未用的页目录项置空，从而大幅节省空间。

$$\text{逻辑地址} = \underbrace{p_1}_{\text{页目录}} \parallel \underbrace{p_2}_{\text{页表}} \parallel \underbrace{d}_{\text{偏移}} \quad (5.4)$$

5.3 分段

5.3.1 段表与保护

分段按程序的逻辑结构（代码段、数据段、栈段）把地址空间划分为长度可变的段 (segment)。每个进程一张段表，表项含段的基址 **base** 与界限 **limit**。

$$\text{逻辑地址} = \underbrace{s}_{\text{段号}} \parallel \underbrace{d}_{\text{段内偏移}}, \quad \text{合法} \iff d < \text{limit}[s], \quad PA = \text{base}[s] + d \quad (5.5)$$

若 $d \geq \text{limit}[s]$ 则触发越界异常，实现按段的保护与共享。

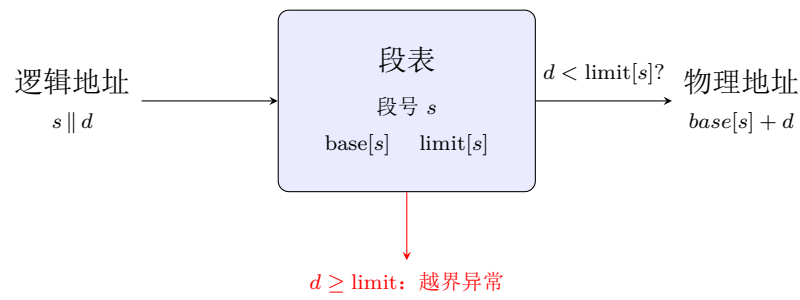


图 5.3: 分段地址翻译与越界检查

5.3.2 分段与分页对比

表 5.3: 分页与分段的对比

维度	分页	分段
划分依据	物理等分为固定大小的页	按逻辑结构划分为变长段
大小	固定（页 = 帧）	可变
地址结构	一维：页号 + 偏移	二维：段号 + 偏移
碎片	仅内部碎片	仅外部碎片
对用户	透明	可见，便于共享与保护

5.3.3 段页式

段页式结合二者：先按逻辑分段，再把每段分页装入物理帧。逻辑地址为三维结构。

$$\text{逻辑地址} = \underbrace{s}_{\text{段号}} \parallel \underbrace{p}_{\text{页号}} \parallel \underbrace{d}_{\text{页内偏移}} \quad (5.6)$$

翻译路径：段表 $[s]$ 得该段的页表 $\rightarrow$ 页表 $[p]$ 得帧号 $f \rightarrow PA = f \times S + d$ 。段页式既保留分段的逻辑与保护，又消除外部碎片；代价是需要多次访存，通常由 TLB 加速。

5.4 TLB 与有效访问时间 EAT

5.4.1 快表 TLB

页表在内存中，若每次访存都先查页表会使访问次数翻倍。转换检测缓冲区 (TLB) 是一块高速相联存储器，缓存最近使用的「页号 → 帧号」表项。

- **TLB 命中**：直接得到帧号，与偏移拼接得物理地址，无需访存页表。
- **TLB 缺失**：访问内存中的页表（可能多级）取帧号，并更新 TLB。

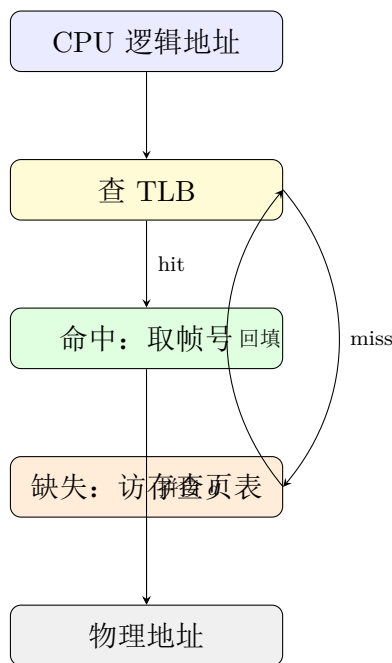


图 5.4: TLB 的命中与缺失路径

5.4.2 有效访问时间

设 TLB 命中率为 h ，TLB 访问时间为 t_{TLB} ，一次内存访问时间为 t_{mem} 。命中时只需「查 TLB + 访存数据」，共 $t_{\text{TLB}} + t_{\text{mem}}$ ；缺失时还需额外一次访存读页表，共 $t_{\text{TLB}} + 2t_{\text{mem}}$ 。

$$EAT = h(t_{\text{TLB}} + t_{\text{mem}}) + (1 - h)(t_{\text{TLB}} + 2t_{\text{mem}}) = t_{\text{TLB}} + (2 - h)t_{\text{mem}} \quad (5.7)$$

若不存在 TLB（每次都查页表），则 $EAT = 2t_{\text{mem}}$ 。可见 $h \rightarrow 1$ 时 $EAT \rightarrow t_{\text{TLB}} + t_{\text{mem}}$ ，接近直接访存的开销。

表 5.4: EAT 随 TLB 命中率的变化 ($t_{\text{TLB}} = 10 \text{ ns}$, $t_{\text{mem}} = 100 \text{ ns}$)

h	命中路径 (ns)	未命中路径 (ns)	EAT (ns)
0.80	110	210	130.0
0.90	110	210	120.0
0.98	110	210	112.0
1.00	110	210	110.0

结论：TLB 命中率每提升一点都会显著降低平均访存时间，因此命中率与 TLB 容量、置换策略密切相关；多级页表虽然省空间，却增加了缺失时的访存次数，二者需与 TLB 配合折中。

第六章 虚拟内存

虚拟内存让每个进程都以为自己独占一整块连续地址空间：它把内存抽象成「按需装入的页」，用磁盘充当后备存储，并以页面置换让有限的物理帧在多个进程间复用。本章沿「按需分页 → 置换策略 → Belady 异常 → 抖动与工作集」的主线展开，重点能用可计算的引用串手算缺页次数。

6.1 按需分页与缺页处理流程

虚拟地址被划分为页号与页内偏移，由 MMU 借助页表把页号翻译成物理帧号。按需分页的核心是：进程启动时并不把全部页调入内存，只有当指令或数据真正被访问、且对应页不在内存时，才触发缺页异常把页调入。这种「懒惰装入」显著减小了驻留集与启动开销。

6.1.1 地址翻译与页表项

页号 p

$v-p$ 位

$\parallel$

页内偏移 d

p 位

$\xrightarrow{\text{页表}}$

帧号 f

物理地址高位

$\parallel$

d

(6.1)

页表中的每个页表项（PTE）除了帧号，还要携带若干状态位来支撑按需分页与置换：

表 6.1: 页表项的典型字段		
字段	含义	作用
有效位 (valid)	该页是否在内存	为 0 时访问即触发缺页
帧号 (frame)	物理帧编号	地址翻译的结果
脏位 (dirty)	调入后是否被写过	决定换出时是否写回磁盘
访问位 (reference)	近期是否被访问	供 Clock/LRU 近似算法使用
保护位 (protect)	读/写/执行权限	越权访问产生保护异常

6.1.2 缺页处理流程

触发缺页后，硬件先陷入内核，再由缺页处理程序完成调页。整个流程可概括为六步：**trap** → **查页表** → **判定缺页** → **调页入内存** → **更新页表** → **重新执行指令**。

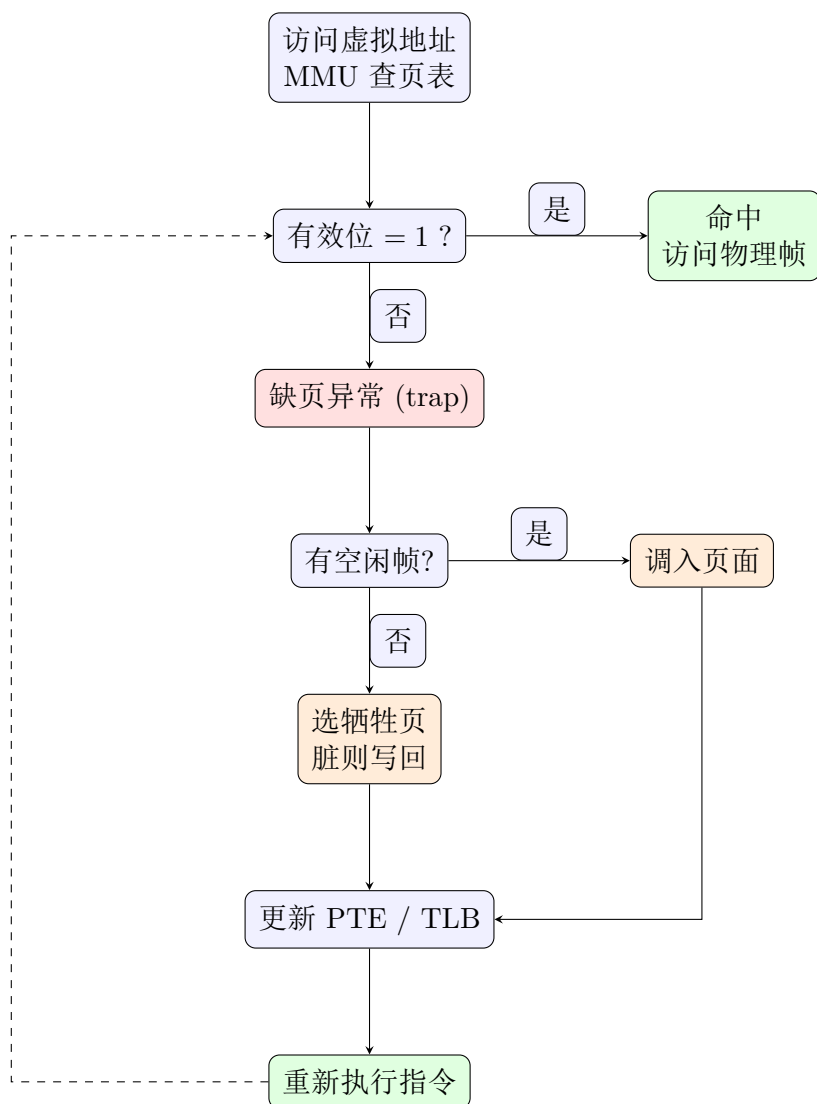


图 6.1: 按需分页的缺页处理流程：查表 → trap → 调页 → 更新 → 重执行

缺页会带来磁盘 I/O，其代价远高于一次内存访问。设缺页率为 p ，则**有效访问时间**（EAT）为内存命中与缺页处理的加权平均：

$$EAT = (1 - p)t_{\text{mem}} + pt_{\text{fault}}, \quad p = \frac{F}{N} \quad (6.2)$$

其中 F 为缺页次数， N 为总访问次数。 t_{fault} 通常在毫秒量级，比 t_{mem} （约百纳秒）大四个数量级，因此缺页率哪怕只有百分之几都会让性能急剧下降——这正是置换算法与工作集管理要解决的问题。

6.2 页面置换算法：FIFO / LRU / 最优 OPT

物理帧有限，当缺页且无空闲帧时必须换出某一页。选择牺牲页的规则即页面置换算法。评价指标是给定引用串下的缺页次数。本节以经典引用串 20 次访问为例：

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

6.2.1 三种算法的直觉

- **FIFO**：淘汰最早进入内存的页，用队列实现；简单，但可能换出仍频繁使用的页，且存在 Belady 异常。
- **LRU**：淘汰最久未被使用的页，符合局部性；需要记录访问时序，硬件完全实现代价高，常用 Clock 近似。
- **OPT**（最优）：淘汰未来最长时间不会被访问的页；缺页次数理论下界，但需预知未来，仅作参考。

6.2.2 帧数 = 3 的逐步模拟

下表给出三种算法在相同引用串、相同 3 个物理帧下的逐步状态（“—”表示空帧）。

表 6.2: FIFO（先进先出），共缺页 15 次					
步	访问页	帧 1	帧 2	帧 3	结果
1	7	7	—	—	缺页
2	0	7	0	—	缺页
3	1	7	0	1	缺页
4	2	2	0	1	缺页（换出 7）
5	0	2	0	1	命中
6	3	2	3	1	缺页（换出 0）
7	0	2	3	0	缺页（换出 1）
8	4	4	3	0	缺页（换出 2）
9	2	4	2	0	缺页（换出 3）
10	3	4	2	3	缺页（换出 0）
11	0	0	2	3	缺页（换出 4）
12	3	0	2	3	命中
13	2	0	2	3	命中
14	1	0	1	3	缺页（换出 2）
15	2	0	1	2	缺页（换出 3）
16	0	0	1	2	命中
17	1	0	1	2	命中
18	7	7	1	2	缺页（换出 0）
19	0	7	0	2	缺页（换出 1）
20	1	7	0	1	缺页（换出 2）

表 6.3: LRU (最近最久未使用), 共缺页 12 次

步	访问页	帧 1	帧 2	帧 3	结果
1	7	7	—	—	缺页
2	0	7	0	—	缺页
3	1	7	0	1	缺页
4	2	2	0	1	缺页 (换出 7)
5	0	2	0	1	命中
6	3	2	0	3	缺页 (换出 1)
7	0	2	0	3	命中
8	4	4	0	3	缺页 (换出 2)
9	2	4	0	2	缺页 (换出 3)
10	3	4	3	2	缺页 (换出 0)
11	0	0	3	2	缺页 (换出 4)
12	3	0	3	2	命中
13	2	0	3	2	命中
14	1	1	3	2	缺页 (换出 0)
15	2	1	3	2	命中
16	0	1	0	2	缺页 (换出 3)
17	1	1	0	2	命中
18	7	1	0	7	缺页 (换出 2)
19	0	1	0	7	命中
20	1	1	0	7	命中

表 6.4: 最优置换 OPT, 共缺页 9 次 (理论下界)

步	访问页	帧 1	帧 2	帧 3	结果
1	7	7	—	—	缺页
2	0	7	0	—	缺页
3	1	7	0	1	缺页
4	2	2	0	1	缺页 (换出 7)
5	0	2	0	1	命中
6	3	2	0	3	缺页 (换出 1)
7	0	2	0	3	命中
8	4	2	4	3	缺页 (换出 0)
9	2	2	4	3	命中
10	3	2	4	3	命中
11	0	2	0	3	缺页 (换出 4)
12	3	2	0	3	命中
13	2	2	0	3	命中
14	1	2	0	1	缺页 (换出 3)
15	2	2	0	1	命中
16	0	2	0	1	命中
17	1	2	0	1	命中
18	7	7	0	1	缺页 (换出 2)
19	0	7	0	1	命中
20	1	7	0	1	命中

表 6.5: 三种算法缺页次数对比（引用串 20 次，帧数 3）			
算法	缺页次数	缺页率	备注
FIFO	15	75.0%	实现最简单，可能产生异常
LRU	12	60.0%	贴近局部性，硬件实现昂贵
OPT	9	45.0%	理论最优，不可在线实现

6.3 Belady 异常

直觉上增加物理帧应当减少缺页。但对 **FIFO**，存在引用串使**帧数增多、缺页反而增多**，这一反直觉现象称为 **Belady 异常**。经典反例引用串为：

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

表 6.6: FIFO 在 3 个帧下缺页 9 次			
步	访问页	帧	结果
1	1	1	缺页
2	2	1 2	缺页
3	3	1 2 3	缺页
4	4	4 2 3	缺页（换出 1）
5	1	4 1 3	缺页（换出 2）
6	2	4 1 2	缺页（换出 3）
7	5	5 1 2	缺页（换出 4）
8	1	5 1 2	命中
9	2	5 1 2	命中
10	3	5 3 2	缺页（换出 1）
11	4	5 3 4	缺页（换出 2）
12	5	5 3 4	命中

表 6.7: FIFO 在 4 个帧下缺页 10 次（比 3 帧更多，即 Belady 异常）

步	访问页	帧 1	帧 2	帧 3	帧 4
1	1	1	—	—	—
2	2	1	2	—	—
3	3	1	2	3	—
4	4	1	2	3	4
5	1	1	2	3	4
6	2	1	2	3	4
7	5	5	2	3	4
8	1	5	1	3	4
9	2	5	1	2	4
10	3	5	1	2	3
11	4	4	1	2	3
12	5	4	5	2	3

结论：FIFO 会发生 Belady 异常，因为它**不具备栈性质**。若把 n 个帧的驻留集合记为 $S_n(t)$ ，栈性质要求 $S_n(t) \subseteq S_{n+1}(t)$ （任何时刻小内存的页集合是大内存页集合的子集）。**LRU** 与 **OPT** 都是栈式算法，因此永不发生 Belady 异常，缺页次数随帧数单调不增。

$$\text{栈性质: } S_n(t) \subseteq S_{n+1}(t) \implies F_n(t) \geq F_{n+1}(t) \quad (6.3)$$

6.4 抖动与工作集模型

6.4.1 抖动（Thrashing）的成因

当多道程序度过高、进程的工作集之和超过可用物理帧总数时，会发生连锁反应：进程频繁缺页 → 内核忙于换入换出 → 就绪队列中进程长期得不到完整驻留 → 缺页更频繁。此时 CPU 大部分时间在等待磁盘，实际有效利用率反而骤降，这种现象称为抖动。

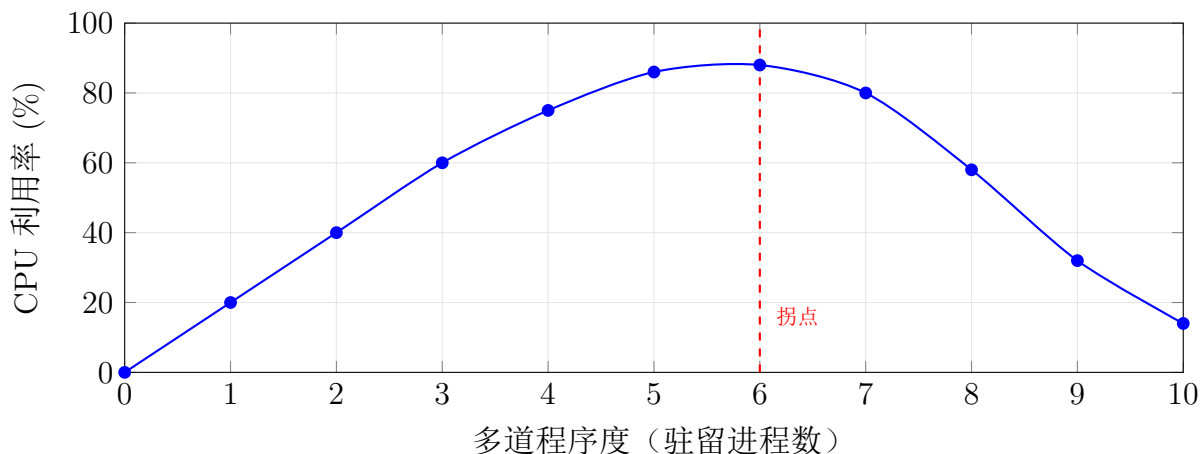


图 6.2: CPU 利用率随多道程序度先升后降，下降段即抖动区

6.4.2 工作集模型

工作集模型用「最近一段时间内被访问的页」来刻画进程的**局部性**。给定窗口大小 Δ （以最近 Δ 次内存访问计），定义时刻 t 的工作集：

$$W(t, \Delta) = \{ \text{最近的 } \Delta \text{ 次访问中出现过的页} \} \quad (6.4)$$

工作集大小 $|W(t, \Delta)|$ 是进程在窗口内真正需要的帧数。系统的**局部性**体现在：

- **时间局部性**：刚被访问的页很可能马上又被访问，故 LRU 有效。
- **空间局部性**：相邻地址的页常被连续访问，故预取与页大小选择有意义。

据此可判断抖动并给出分配准则：

$$\sum_i |W_i(t, \Delta)| > M \implies \text{抖动} \quad \text{分配准则: 帧}_i \geq |W_i(t, \Delta)| \quad (6.5)$$

其中 M 为全部物理帧数。若各进程工作集之和超过 M ，则必须挂起或换出（减少多道程序度）部分进程，直到剩余进程都能获得不小于其工作集的帧数，抖动才能消退。窗口 Δ 太小会低估需求、太大又会把不相关的历史页计入，实践中常配合缺页率（PFF）动态调整。

$$\text{缺页率 } p = \frac{F}{N} : \quad p > p_{\text{upper}} \Rightarrow \text{加帧}, \quad p < p_{\text{lower}} \Rightarrow \text{减帧} \quad (6.6)$$

第七章 文件系统

文件系统把磁盘抽象为「命名空间-文件-目录」的层次结构，统一管理块的分配、元数据与崩溃恢复。

本章围绕磁盘布局、inode 多级索引、分配方式、空闲空间管理与日志一致性展开。

7.1 文件系统布局

磁盘上的文件系统通常划分为若干连续区域。以类 UNIX 的 `ext` 系列为例，自低地址起依次为引导块、超级块、inode 位图、数据位图、inode 表与数据块区。

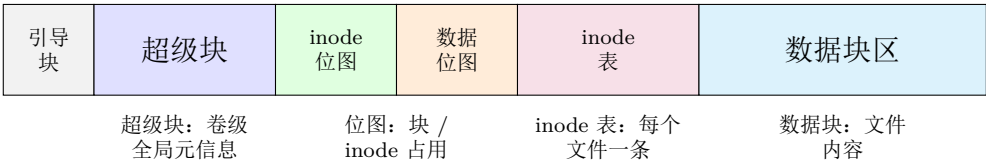


图 7.1: 文件系统的磁盘布局示意

表 7.1: 主要区域的职责

区域	内容	作用
超级块 (superblock)	块数、块大小、空闲块数、inode 总数、magic	挂载时读取，描述整个卷
inode 区 (inode table)	每文件一条 inode (权限、大小、时间、块指针)	文件元数据与数据定位
数据块区 (data blocks)	文件内容本身	真正存放用户数据
位图 (bitmap)	每块 / 每 inode 一位	记录分配与占用状态

超级块是卷的「身份证」：挂载时内核读取它即可获知块大小、空闲块总数与 inode 数量。若超级块损坏，整个卷将无法正确解析，因此实际系统会保存多份副本。

磁盘地址 = 块号 × 块大小 + 块内偏移

(7.1)

7.2 inode 与多级索引

inode (index node) 是文件元数据的集合。它不包含文件名（文件名保存在目录项中），而是记录文件属性与指向数据块的指针。为在固定大小的 inode 中支持大文件，采用多级索引：

- 直接指针 (direct): inode 内含若干直接块号，直接指向数据块，命中时只需一次读盘。
- 一级间接 (single indirect): 指向一个指针块，块内每项再指向一个数据块。
- 二级间接 (double indirect): 指针块 → 指针块 → 数据块，容量为 p^2 。
- 三级间接 (triple indirect): 再增一层，容量为 p^3 ，支持超大文件。

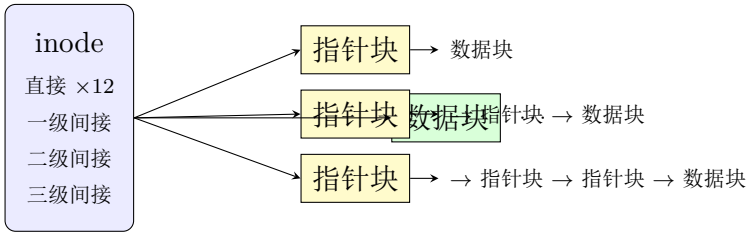


图 7.2: inode 的直接与多级间接指针

设块大小为 b 、指针大小为 s ，则每块可容纳指针数 $p = \lfloor b/s \rfloor$ ；inode 含 d 个直接指针。文件的最大长度由各层容量求和得到：

$$B_{\max} = (d + p + p^2 + p^3) \cdot b, \quad p = \left\lfloor \frac{b}{s} \right\rfloor \quad (7.2)$$

表 7.2: 多级索引容量分解 ($b = 4096$ B, $s = 4$ B, 则 $p = 1024$, $d = 12$)

层次	块数	容量	说明
直接	$d = 12$	≈ 48 KB	小文件快速访问
一级间接	$p = 1024$	≈ 4 MB	一次额外读盘
二级间接	$p^2 = 1048576$	≈ 4 GB	中大型文件
三级间接	$p^3 = 1073741824$	≈ 4 TB	超大文件

将各层相加, $b = 4096$ B、 $s = 4$ B、 $d = 12$ 时最大文件约为 4.0 TB。可见 p 越大, 间接层带来的容量增长越显著。

7.3 分配方式

文件数据块在磁盘上的组织方式, 直接决定随机访问与顺序访问的成本。

表 7.3: 连续、链式、索引分配对比

方式	元数据	随机访问	特点
连续分配	起始块 + 长度	$O(1)$ 直接计算	顺序/随机访问都快、实现简单; 外部碎片、扩展难、需预分配
链式分配	首块 + 末块	$O(n)$ 沿链遍历	无外部碎片、易扩展; 随机访问慢、指针占块、链一断即丢
索引分配	索引块	$O(1)$ 查索引后定位	支持随机访问、无外部碎片; 索引块有开销, 大文件需多级/混合索引

$$T_{\text{random}}: \quad \text{连续} = O(1), \quad \text{链式} = O(n), \quad \text{索引} = O(1) \quad (7.3)$$

FAT 文件系统把链式分配的「下一块号」集中存放在**文件分配表**中, 目录项只需记录首块号, 缓解了链断问题, 但随机访问仍需沿表遍历。现代文件系统多以索引 (inode 多级) 为主, 并辅以**区段 (extent)**连续分配, 兼顾顺序吞吐与随机访问。

7.3.1 三种方式的取舍

- **连续**: 适合只读、顺序访问、大小已知的文件; 需处理外部碎片与「洞」。
- **链式**: 适合顺序追加、大小易变的文件; 不适合频繁随机定位。
- **索引**: 适合通用场景; 代价是索引块本身占用空间与一次额外寻道。

7.4 空闲空间管理

文件系统必须高效地记录哪些块空闲。常用两类方法：**位示图与空闲链表**。

- **位示图 (bitmap)**: 每块用一位表示, 0 空闲、1 占用。分配时按字号 + 位号定位, 可用位运算快速找到连续空闲块。
- **空闲链表 (free list)**: 把所有空闲块链成链表, 每块头部存放下一空闲块号; 头插分配与回收均为 $O(1)$ 。
- **成组链接 (grouping)**: 把空闲块分组, 每组第一块记录本组数量与下一组首块号, UNIX 采用此法, 兼顾空间开销与批量分配。

表 7.4: 空闲空间管理方法对比

方法	优点	缺点
位示图	简单、可随机查找、易找连续空闲块	空间随磁盘线性增长; 找首块可能需扫描
空闲链表	几乎零额外空间; 头部分配 / 回收 $O(1)$	难找连续块; 遍历慢; 依赖数据块, 可靠性差
成组链接	空间与效率折中, 可一次分配/回收一批	实现较复杂, 需维护组栈

位示图的开销可以精确计算。设磁盘共 N 块、每块 b 字节, 则位示图占用

$$S_{\text{bitmap}} = \frac{N}{8} \text{ 字节} = \frac{\text{磁盘容量}}{8b} \text{ 字节} \tag{7.4}$$

例如 1 TB 磁盘、4 KB 块: 块数 $N = 2^{40}/2^{12} = 2^{28}$, 位示图仅 $2^{28}/8 = 2^{25}$ 字节 ≈ 32 MB。开销约为磁盘容量的 $1/(8b)$, 通常可以接受。

7.5 日志与崩溃一致性

若在更新元数据的中途掉电, 文件系统可能处于不一致状态。**日志 (journaling)** 通过「先写日志」保证可恢复。

1. **写日志**: 把事务的元数据旧值 / 新值追加写入日志区。
2. **提交 (commit)**: 日志落盘后写入 commit 记录, 事务被标记为「已提交」。
3. **检查点 (checkpoint)**: 把日志中的修改写回数据区的最终位置。
4. **清理**: 检查点完成后回收日志空间。

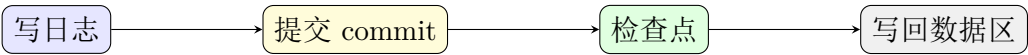


图 7.3: 日志的先写后提交流程

崩溃恢复的依据是「日志中是否存在提交记录」:

表 7.5: 不同崩溃时机的恢复动作

崩溃时机	日志状态	恢复动作
提交前	有记录、无 commit	回滚 / 丢弃该事务，保持旧的、一致状态
提交后、检查点前	已含 commit	重放 (redo) 已提交事务，恢复到新的一致状态
检查点后	修改已写入数据区	无需重放，直接使用已落盘数据

日志可分为：**完全日志**（数据与元数据都记录，恢复最简单，写放大约 2 倍）、**元数据日志**（只记元数据，数据另行有序写，常用折中）以及 **有序写 / WAL** 原则——日志必须先于对应数据落盘，这是恢复正确性的前提。

第八章 I/O 与设备管理

I/O 子系统把五花八门的硬件抽象成统一的读写接口：向上提供设备无关的编程模型，向下通过设备驱动，并借助中断与 DMA 与控制器交互。本章沿「软件层次-I/O 控制方式-磁盘结构-磁盘调度-SSD」展开。

8.1 I/O 软件层次

I/O 软件自上而下分为四层，每一层向上一层提供更抽象、更易用的接口，同时屏蔽下一层的硬件细节。

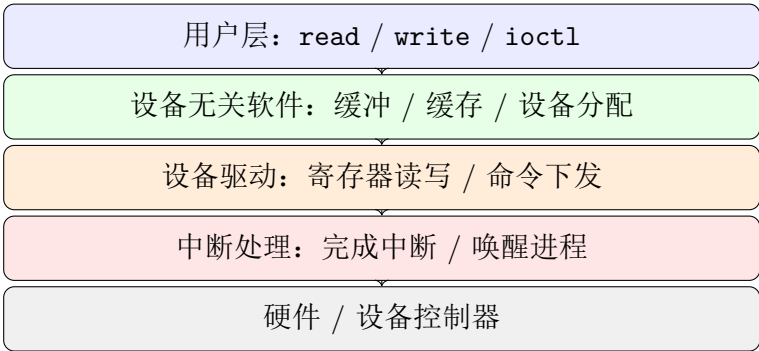


图 8.1: I/O 软件的四层结构

表 8.1: 各层职责

层次	主要职责	典型内容
用户层	发起 I/O 请求，提供面向应用的接口	系统调用 <code>read/write/ioctl</code> 、库函数
设备无关软件	统一抽象与公共策略，屏蔽设备差异	缓冲、缓存、假脱机、设备分配、逻辑名到物理名映射
设备驱动	把抽象请求翻译为设备相关操作	读写控制器寄存器、下发命令、处理设备状态
中断处理	与硬件握手，完成一次 I/O 的收尾	响应完成中断、设置状态、唤醒阻塞进程

用户进程

$\xrightarrow{\text{系统调用}}$

设备无关软件

$\xrightarrow{\text{统一接口}}$

设备驱动

$\xrightarrow{\text{寄存器读写}}$

控制器

(8.1)

关键思想：设备无关软件只依赖统一接口，与具体硬件解耦；设备驱动封装硬件差异，是内核中与设备型号绑定最紧的部分。

8.2 I/O 控制方式

CPU 与设备控制器协同完成一次数据传输，方式有三类：轮询、中断与 DMA。

8.2.1 轮询（程序查询方式）

CPU 不断读取设备状态寄存器，直到设备就绪后亲自搬运数据。实现简单，但 CPU 在等待期间忙等，利用率低，只适合极低速设备。

8.2.2 中断驱动 I/O

设备就绪后主动发出中断，CPU 在中断服务程序中搬运数据。CPU 不必忙等，但每传输一个字/字节都要一次中断，中断开销随数据量线性增长。

8.2.3 DMA（直接存储器访问）

DMA 控制器在设备与内存之间直接成块搬运数据，CPU 只在开始时设置（源、目的、长度）并在结束时收到一次中断。

表 8.2: 三种 I/O 控制方式对比

方式	数据搬运者	CPU 占用	适用场景
轮询	CPU 逐字搬运	全程忙等，几乎 100%	简单、低速、状态可预测的设备
中断	CPU 在中断服务程序中搬运	每字一次中断，开销随数据量增长	中低速、事件随机到达的设备
DMA	DMA 控制器直接搬入内存	仅初始化与结束中断，占用极低	高速、成块传输（磁盘、网卡）

$$T_{DMA} = T_{setup} + \frac{N}{BW} + T_{int}$$

(8.2)

其中 N 为传输字节数， BW 为总线带宽， T_{setup} 为 CPU 设置开销， T_{int} 为结束中断处理开销。当 N 较大时，DMA 的固定开销被摊薄，优势明显。

8.3 磁盘结构与磁盘调度

机械硬盘由盘片、磁头、磁道、扇区与柱面组成。一次随机访问的时间由三部分构成：

$$T_{access} = T_{seek} + T_{rotation} + T_{transfer}, \quad T_{rotation} = \frac{1}{2r}$$

(8.3)

其中 T_{seek} 为寻道时间（磁头移动到目标磁道）， $T_{rotation}$ 为旋转延迟（平均半圈）， r 为转速（转/秒）， $T_{transfer}$ 为传输时间。寻道时间通常远大于其余两项，因此磁盘调度的核心目标是减少寻道长度。

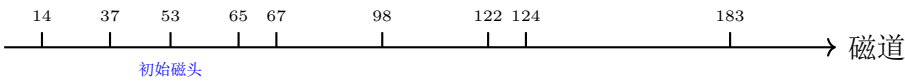


图 8.2: 示例请求序列在磁道轴上的分布（初始磁头位于 53）

8.3.1 常见磁盘调度算法

- FCFS（先来先服务）：按请求到达顺序服务，公平但寻道长度大。
- SSTF（最短寻道时间优先）：每次选距当前磁头最近的请求，平均寻道短，但可能造成远端请求饥饿。
- SCAN（电梯算法）：磁头沿一个方向移动并服务沿途请求，到达该方向最远端后反向；避免饥饿，寻道较均衡。

- **C-SCAN（循环扫描）**：只沿一个方向服务；到达一端后快速返回另一端从头再来，各磁道等待时间更均匀。

表 8.3: 四种磁盘调度算法对比

算法	选择策略	优点	缺点
FCFS	请求到达顺序	公平、简单	寻道长度大，吞吐低
SSTF	距磁头最近	寻道长度小	远端请求可能饥饿
SCAN	单方向扫到底再反向	无饥饿、较均衡	中间磁道服务更频繁
C-SCAN	单方向扫到底再折返	等待时间均匀	折返空跑，额外开销

8.3.2 寻道长度计算示例

设请求序列为 98, 183, 37, 122, 14, 124, 65, 67，初始磁头位于磁道 53。按「向内（磁道号增大）方向优先」计算：

表 8.4: 各算法的服务顺序与寻道长度

算法	服务顺序	寻道长度（磁道数）
FCFS	98,183,37,122,14,124,65,67	640
SSTF	65,67,37,14,98,122,124,183	236
SCAN	65,67,98,122,124,183,37,14	299
C-SCAN	65,67,98,122,124,183,14,37	322

以 FCFS 为例，寻道长度为相邻访问磁头移动距离之和：

$640 = |53 - 98| + |98 - 183| + |183 - 37| + |37 - 122| + |122 - 14| + |14 - 124| + |124 - 65| + |65 - 67| \quad (8.4)$

可见合理调度（SSTF/SCAN）可显著降低总寻道长度，从而提升磁盘吞吐。

8.4 SSD 与闪存

固态硬盘（SSD）基于 NAND 闪存，没有机械部件，随机访问延迟远低于 HDD。其存储层级为页（Page）与块（Block）：

- **页**：读写的最小单位，常见 4KB。
- **块**：擦除的最小单位，通常包含 128 ~ 512 个页。
- **擦除**：闪存只能「先擦后写」，且擦除以块为单位，每块的擦写次数有限。

8.4.1 FTL 与磨损均衡

闪存转换层（FTL，Flash Translation Layer）在逻辑地址与物理页之间做映射，把「覆盖写」转化为「写入新页 + 标记旧页无效」，并在后台执行垃圾回收（GC）。这带来**写放大**：

$$WAF = \frac{\text{实际写入闪存的数据量}}{\text{主机请求写入的数据量}} \geq 1$$

(8.5)

磨损均衡（Wear Leveling）把写入分散到不同物理块，避免个别块提前磨穿；分为动态均衡（只均衡新写入）与静态均衡（也搬移冷数据）。

8.4.2 与 HDD 对比

表 8.5: SSD 与 HDD 对比

维度	HDD	SSD
介质	旋转磁盘 + 磁头	NAND 闪存
随机访问	需寻道与旋转，毫秒级	无机械延迟，微秒级
顺序/随机差异	差异大	差异小
写入特性	可原地覆盖	先擦后写、有写放大
寿命	机械磨损	擦写次数有限，靠磨损均衡
成本/容量	单位容量低	单位容量高

$$T_{\text{SSD}} \approx T_{\text{controller}} + T_{\text{flash read/write}} + T_{\text{GC}}$$

(8.6)

第九章 保护与安全

保护解决「谁（主体）能对什么（客体）做何种操作」的问题：以域为中心建立访问控制，用权限位刻画具体授权，再以保护环、沙箱与虚拟化实现强隔离。

9.1 保护目标与域

保护 (protection) 是操作系统的一种内部机制，用于控制进程与用户对系统资源的访问；**安全 (security)** 则关心系统对来自外部攻击的抵御能力。保护是安全的基础，它回答三个问题：

- **主体 (subject)**：主动发起访问的实体，如进程、线程、用户。
- **客体 (object)**：被访问的资源，如文件、内存段、设备、目录。
- **权限 (right)**：主体可对客体执行的操作，如读、写、执行、追加、删除。

保护域 (protection domain) 是「一组权限」的集合：进程在某一时刻位于某个域中，域规定了它此刻能访问的客体及相应操作。进程可以在运行中**域切换 (domain switching)**，例如通过系统调用以提升或降低权限。

$$\text{域} = \{(\text{客体}, \text{权限集合})\} \quad (9.1)$$

根据主体能力是否可在运行中改变，进程与域的关系有三种模型：

表 9.1: 进程与域的关联模型

模型	含义	例子
静态关联	进程创建到终止始终处于同一域	简单批处理系统
域切换	进程可在多个域间切换（受控）	UNIX 用户态/内核态
动态关联	域的权限集合可被修改	支持最小权限的系统

9.2 访问矩阵、ACL 与能力表

9.2.1 访问矩阵

访问矩阵 (access matrix) 是最一般的保护模型：行是主体，列是客体，矩阵元素 $M[i][j]$ 给出主体 i 对客体 j 的权限集合。

表 9.2: 访问矩阵示例

主体 \ 客体	文件 F_1	文件 F_2	打印机 P
用户 A	读、写	读	—
用户 B	读	—	打印
管理员	读、写、执行	读、写、执行	打印、配置

访问矩阵通常**稀疏**（大多数元素为空），直接按二维数组存储浪费空间，实际系统按行或按列分解实现。

9.2.2 ACL 与能力表

- 访问控制表 (ACL): 按列分解, 每个客体附带一张「哪些主体、有何权限」的列表。检查时先定位客体, 再查主体。
- 能力表 (capability list): 按行分解, 每个主体持有一组「可访问的客体及其权限」的能力 (票证)。能力是不可伪造的令牌, 可直接验证。

表 9.3: ACL 与能力表对比

维度	ACL (按列)	能力表 (按行)
组织方式	每个客体一张主体列表	每个主体一组能力令牌
查询视角	「谁能访问该客体」	「该主体能访问什么」
撤销权限	容易 (改客体列表即可)	困难 (需回收散落的能力)
权限转移	不直接支持	天然支持 (传递能力即可)
典型实现	UNIX 文件权限位、Windows ACL	文件描述符、Kerberos 票据

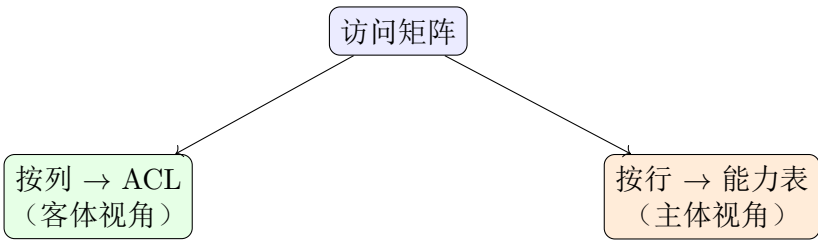


图 9.1: 访问矩阵的行列分解

9.3 UNIX 权限模型

UNIX 把访问矩阵的一行 (文件权限) 压缩为所有者/组/其他三类主体的 3 个权限位, 即经典的 9 位 *rwX* 模型。

9.3.1 *rwX* 与八进制表示

每个文件记录 9 个权限位, 分成三组: 所有者 (owner, *u*)、同组用户 (group, *g*)、其他用户 (other, *o*)。每组三位分别表示读 *r*、写 *w*、执行 *x*。

表 9.4: 权限位与八进制权重

符号	八进制	含义	适用对象
<i>r</i>	4	读: 查看文件内容 / 列出目录	文件与目录
<i>w</i>	2	写: 修改文件 / 在目录增删条目	文件与目录
<i>x</i>	1	执行: 运行程序 / 进入目录	文件与目录

每组的八进制值是该组三位权重之和 (*r*=4, *w*=2, *x*=1), 于是 *rwXr-Xr--* 记为 754:

$$\underbrace{rwx}_7 \quad \underbrace{r-x}_5 \quad \underbrace{r--}_4 \quad \iff \quad 7 = 4 + 2 + 1, \quad 5 = 4 + 1, \quad 4 = 4$$

(9.2)

chmod 既可用符号模式 (chmod *u+x f*), 也可用八进制模式 (chmod 754 *f*)。

9.3.2 目录权限的特殊性

- *r*: 允许列出目录中的文件名 (`ls`)。
- *w*: 允许在目录中创建、删除、重命名条目。
- *x*: 允许进入目录 (`cd`) 并按路径访问其中文件；即使文件本身可读，缺少目录的 *x* 也无法访问。

9.3.3 `setuid`、`setgid` 与 `sticky`

在三位权限之前还可加一位**特殊位**，形成四位八进制表示（如 4755）：

表 9.5: 特殊权限位

位	八进制	作用
SUID (<code>setuid</code>)	4	执行时以文件 所有者 的身份运行（如 <code>/usr/bin/passwd</code> ）
SGID (<code>setgid</code>)	2	文件：以文件 属组 身份运行；目录：新建文件继承该目录的组
Sticky	1	目录中只有文件所有者或 <code>root</code> 才能删除自己的文件（如 <code>/tmp</code> ）

特殊位在符号表示中为 *s*（可执行位存在）或 *S*（可执行位缺失），例如 4755 显示为 `rwsr-xr-x`。

权限判定示例：文件 *F* 权限为 754，属主为 A。用户 B 属于 *F* 的组。B 尝试写入 *F*：组权限为 5 = *r-x*，不含 *w*，故被拒绝，内核返回 `Permission denied (EACCES)`。

9.4 隔离与虚拟化

9.4.1 保护环

CPU 用保护环 (protection ring) 划分特权级，内核位于最内层最高特权环，用户程序位于最外层最低特权环。

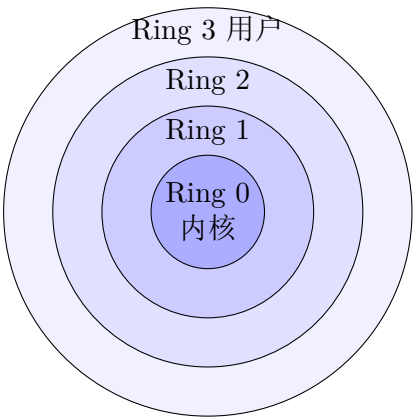


图 9.2: 保护环：特权由内向外递减

表 9.6: 保护环层级

环	权限	典型内容
Ring 0	最高，可执行特权指令	内核、驱动核心
Ring 1	高	部分设备驱动（很少使用）
Ring 2	中	部分驱动（很少使用）
Ring 3	最低	用户应用程序

系统调用是用户态向内核态受控跨越保护环的唯一入口：CPU 通过 trap 切换特权级，验证参数后再执行特权操作。

9.4.2 沙箱

沙箱 (sandbox) 为不可信代码提供受限执行环境，即使代码有恶意也无法突破边界。常见手段：

- 系统调用过滤 (seccomp)：限制进程可用的系统调用集合。
- 命名空间 (namespace)：隔离进程可见的 PID、网络、文件系统挂载等视图。
- 资源限制 (cgroups、rlimit)：限制 CPU、内存、I/O 用量。
- 文件系统重定向 (chroot、jail)：限制根目录或可见路径。

9.4.3 虚拟化与容器

虚拟机 (VM) 通过 Hypervisor 在硬件层抽象出多台虚拟机器，每台运行独立内核；容器 (container) 则共享宿主机内核，靠命名空间与 cgroups 隔离进程视图。

表 9.7: 虚拟化与容器对比

维度	虚拟机	容器	沙箱
隔离层次	硬件级 (Hypervisor)	内核级 (namespace/cgroup)	进程级 (syscall 过滤)
内核	每台独立 Guest 内核	共享宿主内核	共享宿主内核
启动速度	慢 (秒级)	快 (毫秒至秒级)	快
资源开销	大 (完整 OS)	小	最小
隔离强度	强	中	相对弱
典型实现	KVM、Xen、VMware	Docker、containerd	seccomp、jail

归纳：ACL/能力表决定「授权」；*rw**x* 位决定「具体到文件的操作」；保护环、沙箱与虚拟化决定「越权者能走多远」。三者层层递进，共同构成操作系统的保护与安全体系。